

# VCU-LLM: 提示高效 设备端 大型 语言 模型 用于智能家居中的 模糊指令 理解

郑源 张, 北京邮电大学, 中国董 赵\*, 北京邮电大学, 中国天成 何, 北京邮电大学, 中国子龙 王, 北京邮电大学, 中国翔宇 李, 北京邮电大学, 中国华东 马, 北京邮电大学, 中国

智能家居在全球市场正受到越来越多的关注。然而, 当前的智能家居助手系统要么仅通过用户的显式指令控制家电设备, 要么依赖基于云的大语言模型进行模糊指令理解, 这带来了高延迟、隐私担忧和高成本等缺点。在本工作中, 我们提出了VCU-LLM, 这是首个在边缘设备上部署大语言模型以实现本地模糊指令理解和智能设备控制计划生成的系统。VCU-LLM引入了一种新颖的模糊指令知识检索算法, 该算法优化了输入提示中与设备相关的信息, 从而加速了大语言模型的设备端推理并降低了任务复杂度。我们进一步构建了一个用于大语言模型微调的数据集, 以模拟智能家居助手在不同家庭中控制设备的使用场景。在推理过程中, 应用了定制的KV缓存技术以进一步加速推理。我们基于人类评分和LLM评分的评估表明, 与SOTA基线相比, VCU-LLM将生成的控制计划质量平均提升了43.3%, 同时与其他设备端基线相比, 时间开销平均减少了8.44 $\times$ 。我们还通过在一个真实家庭环境中的案例研究实现了VCU-LLM, 证明了其在现实世界应用中的可行性。

CCS 概念: • 以人为中心的计算  $\rightarrow$  普适与移动 计算 理论、概念 与 范式。  
其他关键词和短语: 智能家居环境, 大语言模型, 边缘计算, 模糊指令理解

## ACM 引用格式:

Zhengyuan Zhang, 赵东, Tiancheng He, Zilong Wang, Xiangyu Li, and Huadong Ma. 2026. VCU-LLM: 用于智能家居模糊指令理解的提示高效设备端大语言模型. 《ACM交互式、移动、可穿戴与普适技术汇刊》10, 2, 文章76 (2026年6月), 30 页. <https://doi.org/10.1145/3810190>

## 1 引言

物联网技术的快速发展正在改变人们与物理世界的交互方式。作为日常生活中的一项关键应用, 配备家庭助手的智能家居 [4, 28, 59]

\*通讯作者。

作者联系信息: 张正源, 北京邮电大学, 中国北京, [zhangzhengyuan@bupt.edu.cn](mailto:zhangzhengyuan@bupt.edu.cn); 赵东 (通讯作者), 北京邮电大学, 中国北京, [dzhao@bupt.edu.cn](mailto:dzhao@bupt.edu.cn); 何天成, 北京邮电大学, 中国北京, [wangyuchi@bupt.edu.cn](mailto:wangyuchi@bupt.edu.cn); 王子龙, 北京邮电大学, 中国北京, [2023\\_wzl@bupt.edu.cn](mailto:2023_wzl@bupt.edu.cn); Xiangyu Li, 北京邮电大学, 中国北京, [Lxy011224@bupt.edu.cn](mailto:Lxy011224@bupt.edu.cn); 马华东, 北京邮电大学, 中国北京, [mhd@bupt.edu.cn](mailto:mhd@bupt.edu.cn)。



本 作品 采用 知识共享 署名 4.0 国际 许可协议 进行许可。© 2026 版权 归 所有者/  
作者所有。ACM 2474-9567/2026/6-ART76<https://doi.org/10.1145/3810190>

《ACM交互式、移动、可穿戴与普适技术汇刊》, 第10卷第2期, 文章76。发表日期: 2026年6月。

利用物联网技术实现家用电器的互联互通、远程控制和自动化管理，从而提升用户日常生活的便利性、舒适性和安全性。据 [13], 预测, 全球智能家居市场预计到2025年将达到1740亿美元, 年增长率为9.55%, 并预计到2029年将达到2506亿美元。

在传统的智能家居助手系统中, 理解用户意图依赖于预定义的文本模板[22, 36] 或将特定关键词绑定到相应的设备控制操作 [46, 56]。其他工作[48, 58] 采用机器学习方法处理传统的自然语言理解任务, 例如意图检测和槽位填充, 以解释用户意图。当一条使用的指令包含目标设备名称、期望状态变化以及可选的位置信息时(例如, “关闭客厅的风扇。”), 现有的智能家居助手通常能够生成准确的控制计划。然而, 用户通常更喜欢用模糊指令来表达他们对智能家居设备的控制需求(例如, “帮我降低客厅的温度。”), 以降低交互成本并提升与智能家居助手的整体体验。这些模糊指令缺乏固定的句子模式和特定的关键词, 使得传统方法无法生成令人满意的控制计划。因此, 从模糊指令生成合理控制计划, 需要对可用的智能家居设备信息和模糊指令本身的细微差别有深入的理解。

近年来, 基于Transformer的大语言模型(LLMs) [18, 24, 29, 55] 发展迅速, 在词性标注、意图理解和问答等任务中展现出卓越性能。一些近期研究 [26, 38, 45, 49, 50] 将大语言模型与智能家居环境相结合, 以提供智能服务, 包括模糊指令理解、智能设备的联合控制逻辑生成以及个性化家庭代理。然而, 这些工作大多通过调用云端API来利用大语言模型, 这带来了以下缺点: **(i) 高延迟**。为了减少幻觉并生成准确的控制计划, 现有工作通常需要手动将任务分解为多个子任务, 并进行多次API调用以进行迭代推理。这可能导致高响应延迟, 从而降低用户体验。此外, 响应时间受用户网络状况和服务器实时工作负载(即活跃用户数量)的影响, 使其极不稳定。根据 [38], 使用大语言模型API处理一条模糊指令的平均响应时间超过10秒。 **(ii) 隐私问题**。使用大语言模型API生成控制计划需要上传所有智能设备信息和用户指令。这会增加暴露敏感个人数据的风险, 例如用户的生活习惯、日程安排和家庭详情。 **(iii) 高成本**。大多数大语言模型API根据输入输出令牌的数量收费。根据 [38], 生成每个控制计划的成本在0.02美元(GPT-3.5)到0.03美元(GPT-4)之间。考虑到每位用户每天使用的平均指令数量 [16] 以及当前智能家居市场中的活跃用户数量 [9], 每天至少需要花费3820万美元用于API费用, 这是一笔巨大的开支。此外, 基于云端的大语言模型需要可扩展的计算资源来支持更多并发用户, 这限制了服务的可扩展性。而且, 在没有网络连接的情况下, 智能服务将变得不可用。

为全面解决上述缺陷, 我们提出VCU-LLM, 它将大语言模型部署在家庭内的边缘设备(例如智能音箱和智能家居中枢)上, 以本地执行模糊指令理解并生成控制计划。然而, 一个关键挑战必须解决: 我们如何在资源受限的边缘设备上生成合理控制方案, 同时确保低延迟?

预训练大语言模型在生成控制计划时经常出现幻觉, 例如引用家庭中不存在的设备, 并且频繁产生格式不稳定的控制计划 [33, 54]。若没有适当的任务分解, 大语言模型难以生成合理的控制计划或准确理解用户意图。这一问题在参数较少的大语言模型 [45]中尤为突出。为解决此问题, [38] 将模糊指令理解过程分解为多个子任务, 包括澄清、过滤、规划和执行, 以生成更合理的控制计划。然而, 需要与大语言模型进行多轮推理会显著增加计算开销, 使得该方法不适合资源受限的

边缘设备。当前的智能家居系统通常使用智能音箱或智能家居中枢作为本地控制中心。然而，这些边缘设备通常拥有不到4GB的内存和有限的计算能力，使得在其上部署大参数大语言模型具有挑战性 [66]。例如，即使是一个INT8量化的Llama2-7B模型，推理也至少需要8GB的可用内存。

另一方面，一些研究通过推测解码 [40, 60] 和动态卸载 [19, 30, 51] 等资源调度技术，实现了大参数大语言模型在资源受限的边缘设备上的推理。然而，这些方法通常涉及频繁且耗时的内存I/O操作，难以满足用户的延迟要求。即使是轻量化大语言模型（例如，具有10亿参数的模型），其推理速度也可能较低，尤其是当输入提示包含家庭中所有智能设备的全面信息时。

为了解决上述挑战，我们首先设计了一种轻量级**模糊指令知识检索算法**，用于提炼输入提示中与设备相关的信息，从而提升大语言模型的设备端推理速度。我们将完整的输入提示分为三个部分：(i) 用户的指令，(ii) 设备的可用服务（例如，灯可以开启/关闭、改变亮度、调整颜色），以及 (iii) 设备的当前状态（例如，窗帘当前是关闭的）。我们观察到，不同的指令对这三个部分有不同的需求，以生成合理的控制计划。有些指令不需要任何设备相关信息（(ii) 和 (iii)）即可产生响应（例如，“今天天气怎么样？”），我们将其视为失败案例）。另一些可能不需要设备的当前状态（例如，“我要睡觉了。”），而有些则需要访问所有三个部分才能生成合理的控制计划（例如，“现在风太小了。”）。为了实现这一点，我们基于预训练句子转换器 [47] 设计了一个轻量级分类器，以确定对于给定的指令，输入提示的哪些部分是必需的。此外，我们对家庭内的可控因素（例如，照明、温度、声音）进行了分类，并从用户的模糊指令中提取特征，将其映射到用户意图调整的具体因素。基于此映射，我们从所有可用的智能设备中筛选出相关设备。提供明确的控制因素和相应的候选设备，有助于降低大语言模型生成控制计划的复杂性，并减轻幻觉问题。

然而，上述对大语言模型输入提示的改进，仍未能完全解决输出格式不稳定以及对模糊指令下用户意图理解有限的问题，这对于能力受限的轻量化大语言模型尤为突出。为解决此问题，我们提出了一种模糊指令控制数据集生成方法，该方法构建了一个涵盖多样化模糊指令、智能设备类型和家庭房间的数据集，从而模拟了不同环境下真实世界的智能家居助手使用场景。随后，我们应用指令微调，利用此数据集将知识从一个泛化能力良好的大语言模型（例如GPT-4）提炼到适合部署在边缘设备上的轻量化大语言模型中。在设备端推理过程中，我们观察到，一旦识别出家庭中的可用设备，在执行不同用户指令时，输入提示内容的某些部分将保持不变。基于这一观察，我们将来自设备端推理引擎 [7] 的KV缓存技术 [42, 44, 63] 与受限的输出令牌相结合，以在边缘设备上实现额外的推理加速。

本工作的主要贡献如下：

- 我们提出了VCU-LLM，这是首个用于智能家居环境中设备端模糊指令理解与设备控制计划生成的方法。它包括：(1) 一个用于模糊指令的模糊指令知识检索模块，用于约束输入提示并缓解幻觉；(2) 一种用于构建模糊指令数据集并执行后续大语言模型微调的可扩展策略；以及(3) 一种带有KV缓存管理的设备端推理机制，以平衡延迟和内存开销。
- 我们通过人工评分（招募了36名参与者，每人完成171项评分任务）和基于大语言模型的大规模评分，对VCU-LLM的性能进行了综合评估。生成的控制计划质量平均提升了43.3%，而时间开销则

系统指令: **[指令]** (例如  
“你是一个控制智能家居的有用AI助手。你接收你的命令并为设备分配设置作为回应。  
用户指令、来自可用设备的可控服务（一个JSON格式的列表）以及设备当前状态（一个列表）在下方提供。  
如果有与用户指令相关的设备，请用设备JSON（相关设备仅）以以下形式分配已更改设置: {status : success, devices: {}, explanation: "" }  
如果没有与用户指令相关的设备，请以以下JSON形式响应: {status: 失败}")

服务: **[JSON]** (例如  
[{"id": 1, "area": "客厅", "type": "灯", "services": {"light": {"on": {"description": "开关状态", "format": "bool", "access": ["read", "write"]}, "set\_brightness": {"description": "亮度", "format": "uint8", "access": ["read", "write"], "unit": "百分比", "value-range": [1, 100, 1]}, "set\_color": {"description": "色温", "format": "uint16", "access": ["读取", "写入"], "unit": "开尔文", "value-range": [2700, 6500, 1]}}}], ...)

当前状态: **[JSON]** (例如 [{"客厅灯": "开启", "亮度": 81, "颜色": 4025}], ...)

用户指令: **[指令]** (例如 房间太冷了。)

图1. 用于大语言模型生成设备控制计划的输入提示文本。

平均降低了 8.44×，相较于其他使用轻量化大语言模型（例如HomeLLM [6], Llama3.2-1B [25] 和 TinyLlama-1.1B [65]）的设备端基线。

- 我们在现实世界中部署了VCU-LLM，进行了一项评估整体系统性能的案例研究。我们还构建了一个数据集，包含超过40,000个跨越不同家庭环境的高质量模糊指令-控制计划对，用于智能家居设备控制场景下的大语言模型微调。我们的数据集已公开可用<sup>1</sup>，代码也将很快发布。

## 2 背景与动机

在本节中，我们进行了初步实验，以展示第1节所述挑战的影响。

### 2.1 大语言模型在智能家居设备控制计划生成中的性能

为了测试现有大语言模型在设备控制计划生成方面的性能，我们首先解释大语言模型如何利用智能家居设备的信息。然后，我们设计了一项用户调查，由参与者手动评估大语言模型生成的控制计划的质量。最后，我们分析调查结果，以识别当前基于LLM的方法的局限性，并为提升控制计划质量提供见解。

**2.1.1 将大语言模型引入智能家居环境。**我们构建了一个工作流程，利用大语言模型配合零样本提示词，在智能家居环境中响应用户指令生成设备控制计划。根据先前关于智能家居中基于LLM的控制计划生成的研究 [26, 38, 57]，输入提示词通常包含几个组成部分，如图1所示。给定提示词后，大语言模型会生成一个JSON控制计划，该计划将用户指令中推断出的目标映射为实现该目标所需的具体设备设置变更。为了测试基于LLM方法的不同设置和模型的性能，我们引入了以下6个基线工作流程：

<sup>1</sup><https://www.kaggle.com/datasets/liema77/on-device-vcu-llm-vague-smart-home-commands>

• **Llama-7B [55]:** 一个中等参数规模的预训练大语言模型，部署在配备A100 GPU的云服务器上。该模型使用图1所示的提示词模板进行单次推理以生成控制计划。

• **HomeLLM-1.1B [6]:** 一个参数规模较小的大语言模型，首先进行预训练，然后使用家庭助手请求数据集进行微调 [5]（采用LoRA技术） [32]。该微调数据集包含一系列指令、设备信息以及与负责管理家庭所有智能设备的智能家居助手交互的相应回复。然而，该数据集中的大部分指令都是显式指令（包含目标设备名称、状态变更和位置信息）。该模型已部署，并在与Llama-7B模型相同的服务器上，使用图1所示的相同提示模板进行单次推理，以生成控制计划。

• **GPT-4o 与 GPT-3.5 [18]:** 由OpenAI提供的两种泛化能力强的商用大语言模型 [10]。这两个模型通过OpenAI API调用 [11]，使用图1所示的相同提示模板进行单次推理。API费用根据输入提示词元和生成令牌的数量计费。

• **GPT-4o (I) 与 GPT-3.5 (I):** 使用迭代推理的相同GPT模型。根据 [38]，我们将任务分解为：(i) **澄清**：判断用户指令的目标是否可通过现有设备实现。(ii) **过滤**：过滤掉与用户指令无关的设备。(iii) **规划**：仅使用相关设备生成控制计划。(iv) **映射**：根据可用设备的服务，将生成的控制计划转换为JSON格式。我们通过多次顺序调用大语言模型API来执行这些子任务，针对每个子任务使用不同的提示词并结合前序步骤的输出。迭代推理未应用于Llama-7B和HomeLLM-1.1B，因为这两个模型的输出内容和格式不稳定，不适合执行连续的子任务。

**2.1.2 用户调查设计。**为了评估上述基于LLM的方法的性能，我们进行了一项用户调查，招募参与者通过问卷对生成的控制计划进行手动评分。

**智能家居环境设计。**为了让大语言模型能够生成设备控制计划，我们构建了一个智能家居环境——一个包含多个常见家庭房间（如客厅、卧室和厨房）的家庭模板，如图2所示。每个房间都配备了符合用户日常习惯的典型家电设备。我们还为每个设备分配了当前工作状态。此家庭环境中所有设备的可用服务和当前状态都被整合到用于设备控制计划生成的输入提示中。

**问卷设计。**为了全面评估基于LLM的方法的性能，我们设计了20条模糊指令，如图3所示。我们使用上述6种基于LLM的方法以及设备信息，为每条模糊指令生成6种不同的控制计划。为了让参与者能在问卷中准确评估所生成控制计划的质量，我们在问卷开头展示了图2以及每个设备的可用服务（以JSON格式），确保参与者充分理解当前的智能家居环境。随后，问卷中列出了针对每条模糊指令的六种不同控制计划。参与者被要求根据图2所示的环境，对智能家居助手提供的生成控制计划标注“满意度”（从1分最不满意到5分最满意）。如果某个控制计划得分低于5分，参与者需用自己的话简要描述不满意的原因。为确保参与者在填写问卷时不受无关因素影响，我们对用于生成控制指令的模型信息进行了匿名处理，并禁止参与者之间交流。每位参与者需要完成总计120项评分任务，耗时约60分钟。在控制计划生成过程中，我们将所有大语言模型的温度参数设置为较低值（例如0.2），以确保大语言模型输出更一致的内容。

**参与者。**我们通过线下联系招募了10名参与者。参与者年龄在23至27岁之间，其中1名女性，9名男性。所有参与者都熟悉智能家居场景，并且能够理解智能家居助手的工作流程。

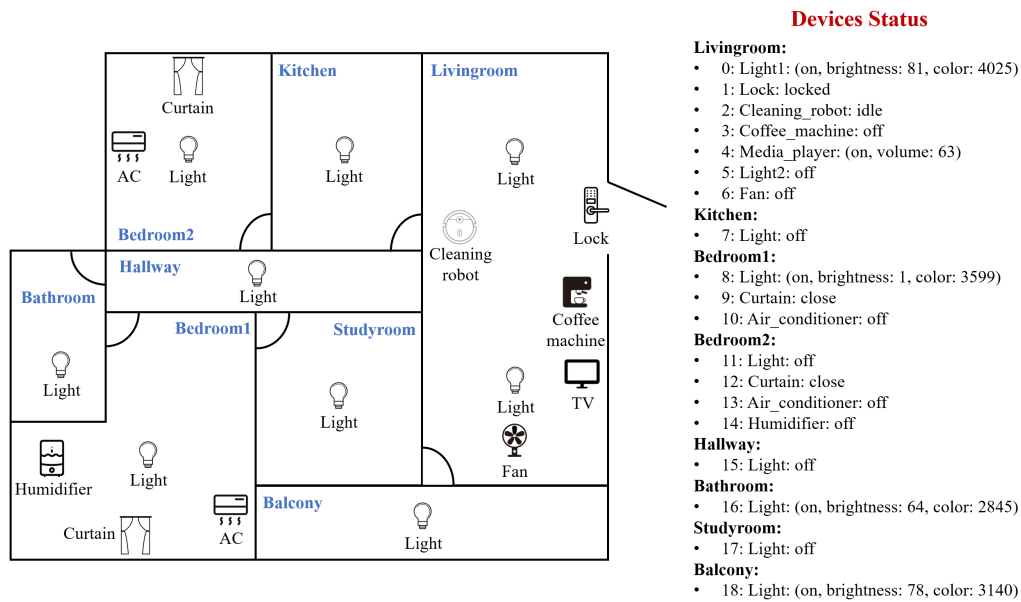


图2. 一个包含设备布局 and 当前状态 of 智能家居环境，用于评估基于大语言模型的方法在生成控制计划方面的性能。

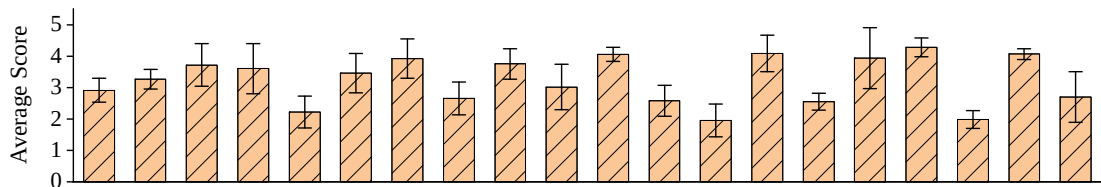


图3. 针对每条模糊指令所生成控制计划的平均得分。误差棒代表在不同参与者间计算得出的标准差。

2.1.3 用户调查分析。在本小节中，我们基于问卷结果分析了现有基于大语言模型方法的性能。我们计算了10名参与者对每种基于大语言模型方法的平均评分（如图4所示），记录了平均延迟（如图5所示），并呈现了20条模糊指令各自的平均得分（如图3所示）。现在，我们从结果中列出一些发现。

(i) 中型LLM通常难以根据输入提示中的任务要求生成正确的内容或格式。如图4所示，Llama-7B获得了最低的平均分2.04。即使提示中明确指定了执行任务的步骤，Llama-7B仍然无法产生合理的响应（如图6所示）。

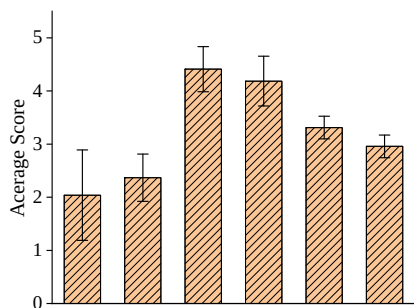


图 4. 不同基于大语言模型的方法在20条模糊指令上的平均得分。误差棒代表根据不同参与者计算出的标准差。

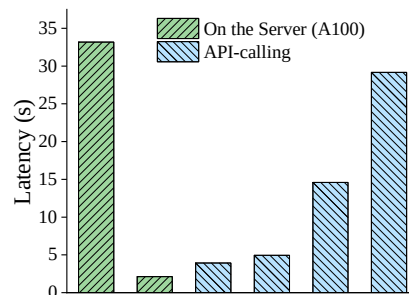


图 5. 不同基于大语言模型的方法生成每条控制计划的平均延迟。

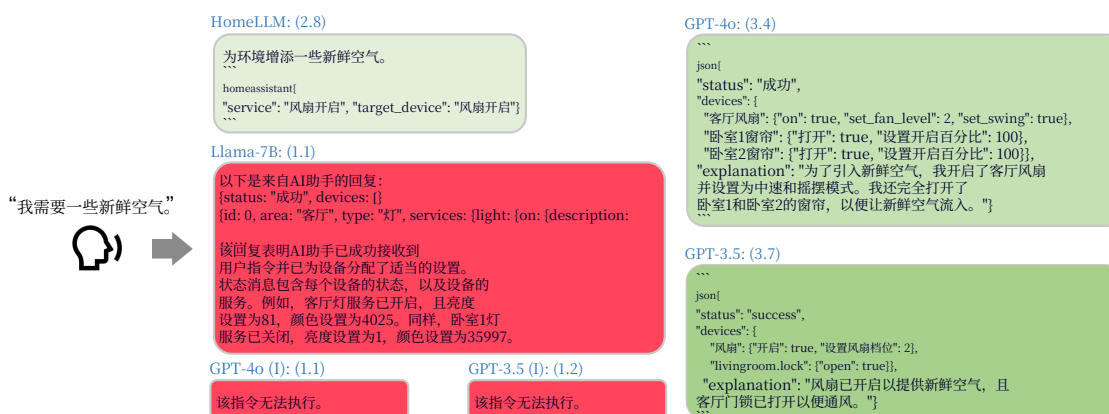


图6. 输出“气”的输出内容与平均得分。

sh

(ii) 即使对于性能最高的商用大语言模型，准确识别与模糊指令相关的设备仍然具有挑战性。如图4所示，GPT-4o获得了最高分4.41。然而，在处理某些模糊指令时，GPT-4o会生成包含无关设备的控制计划。例如，在处理指令“房间太冷了。”时，GPT-4o开启了风扇（该设备不具备制热功能），而不是启动空调并将其设置为制热模式。如图6所示，在处理指令“我需要一些新鲜空气”时，GPT-4o打开了窗帘（误以为是开窗），这也是不相关的。

(iii) 在任务相关数据上进行微调的大语言模型展现出显著的性能提升。如图4所示，仅使用11亿参数，微调后的HomeLLM就获得了比Llama-7B更高的平均分。与Llama-7B相比，HomeLLM-1.1B能生成更一致的输出格式，并包含清晰的控制因素，如图6所示。然而，HomeLLM-1.1B存在幻觉问题（例如，

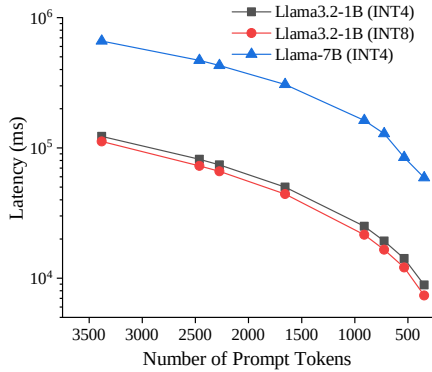


图 7. 不同提示词长度下的提示延迟。

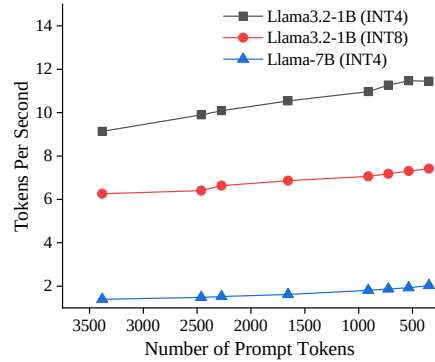


图 8. 不同提示词长度下每秒生成的令牌数量。

生成了不存在的设备，如“睡眠”或“气候”）。此外，微调数据集仅包含显式指令，这限制了 HomeLLM-1.1B 在模糊指令上的整体性能。

(iv) **通过迭代推理生成的控制计划往往更为保守。**如图 4 和图 6 所示，使用 GPT-4o 和 GPT-3.5 进行迭代推理所得的分数甚至低于单次推理。这是因为，在判断一条指令能否执行时，部分模糊指令由于与家庭中可用的设备服务不完全匹配，会被判定为无法实现。

(v) **通过 API 调用利用大语言模型会导致延迟不稳定。**如图 5 所示，使用 API 调用时，通过单次推理生成一个控制计划的平均时间为 4.43 秒，而使用迭代推理时则增加到 21.87 秒。我们观察到，API 调用的延迟受网络状况和并发用户数量的双重影响，导致全天的延迟极不稳定。

## 2.2 资源受限的边缘设备上大语言模型的推理延迟

为了评估在资源受限的家庭边缘设备上本地运行的大语言模型的推理延迟，我们在一个树莓派 [25] 设备上部署了三种不同的大语言模型（采用 INT4 量化的 Llama3.2-1B、采用 INT8 量化的 Llama3.2-1B，以及采用 INT4 量化的 Llama-7B）。然后，我们使用图 5 [12] 2 中所示的设备信息作为输入提示，对上述模型执行一次性本地推理。树莓派 5 是一款低功耗边缘设备，功耗为 27W，配备 2.4GHz Arm Cortex-A76 CPU 和 8GB 内存。其价格低于 80 美元，为边缘开发者提供了极具成本效益的选择。所有在树莓派 5 [12] 上运行的大语言模型均使用 Llama.cpp [7] 推理框架。

如图 7 和图 8 所示，我们通过每次移除一个房间内的所有设备信息，逐步缩短输入提示的长度。我们记录了处理提示所需的时间（即生成第一个词元的时间）以及每秒生成的令牌数量。我们观察到：

(i) **设备信息导致输入提示中的令牌数量过多。**服务和当前状态部分由于详细的规格说明（例如，数据格式、类型、取值范围）占据了令牌使用的主导地位。如图 2 所示，当包含所有设备时，输入提示达到 3,382 个令牌，引入了显著的冗余。

(ii) **处理输入提示占据了推理时间的很大一部分。**如图 7 所示，对于 Llama3.2-1B (INT8)，提示处理需要 7,362 毫秒到 11,904 毫秒，具体取决于词元数量，这对于响应迅速的智能家居交互来说太慢了。在大多数情况下，少于 200 个输出令牌就足以实现

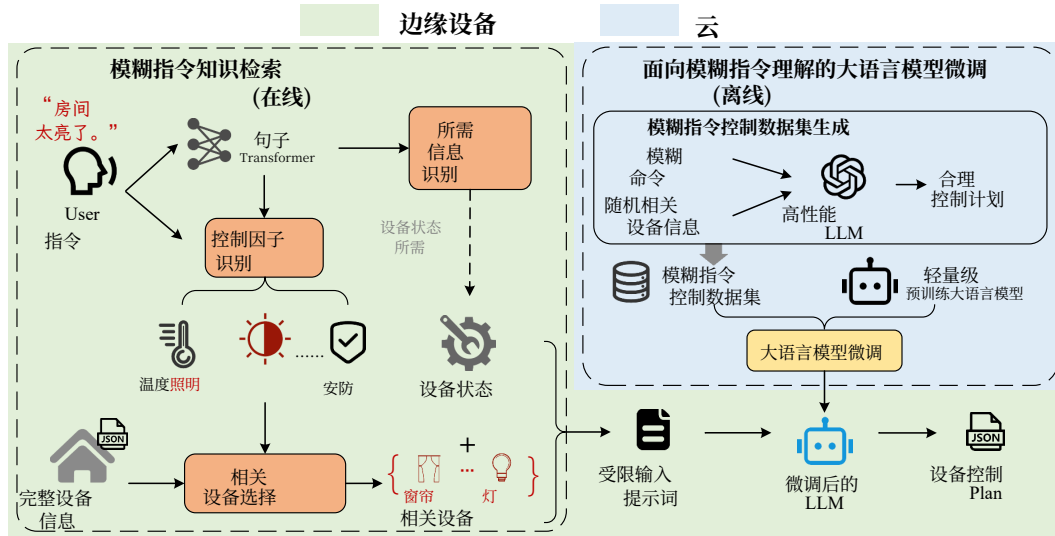


图 9. VCU-LLM 概览。

目标。因此，理解通常需要少于200个输出令牌，从而减少了输入提示的长度对于最小化端到端延迟至关重要。

(iii) 随着提示长度减少，令牌生成速度相应增加。如图8所示，将提示令牌从 3,382 减少到 347，使 Llama3.2-1B (INT4/INT8) 的生成速度分别提升了  $2.31 \times$  和  $1.16 \times$  令牌/秒。然而，在资源受限的边缘设备上，生成速度仍然相对较慢。因此，必须严格规范输出格式以产生简洁的控制计划。此外，对于较小的大语言模型，仅靠提示工程往往因预训练知识和上下文容量有限而无法稳定地约束输出 [34, 35, 45]。因此，有必要使用定制的模糊指令理解数据集进行微调，以增强任务特定能力和推理效率。

(iv) 参数数量庞大的大语言模型（例如 Llama-7B）不适合在边缘设备上本地推理。如图7和图8所示，Llama-7B (INT4) 的提示处理速度平均比 Llama3.2-1B 慢  $6.12 \times$ ，令牌生成速度平均慢  $5.12 \times$ ，无法满足实时执行要求。此外，大型模型会产生较高的内存开销（例如，INT4 量化的 Llama-7B 需要 3.9 GB）和更长的加载时间，从而损害边缘设备高效处理多任务或切换上下文的能力。

### 3 概述

如图9所示，VCU-LLM 包含两个组件：

(i) **模糊指令知识检索**。给定一条用户指令和当前可用的智能家居设备信息（以JSON格式），模糊指令知识检索旨在以轻量级方式生成一个受限输入提示，以提高后续设备端LLM推理的效率。首先，使用预训练的句子转换器将用户指令编码为句子嵌入。然后，在所需信息识别模块中，我们训练一个轻量级分类器来确定所需的设备信息（全部信息、仅服务信息，或无信息）。接着，将句子嵌入和用户指令传递到控制因子识别模块，该模块应用关键词匹配和语义相似性来识别指令意图修改的家庭环境中的因素。最后，基于完整的

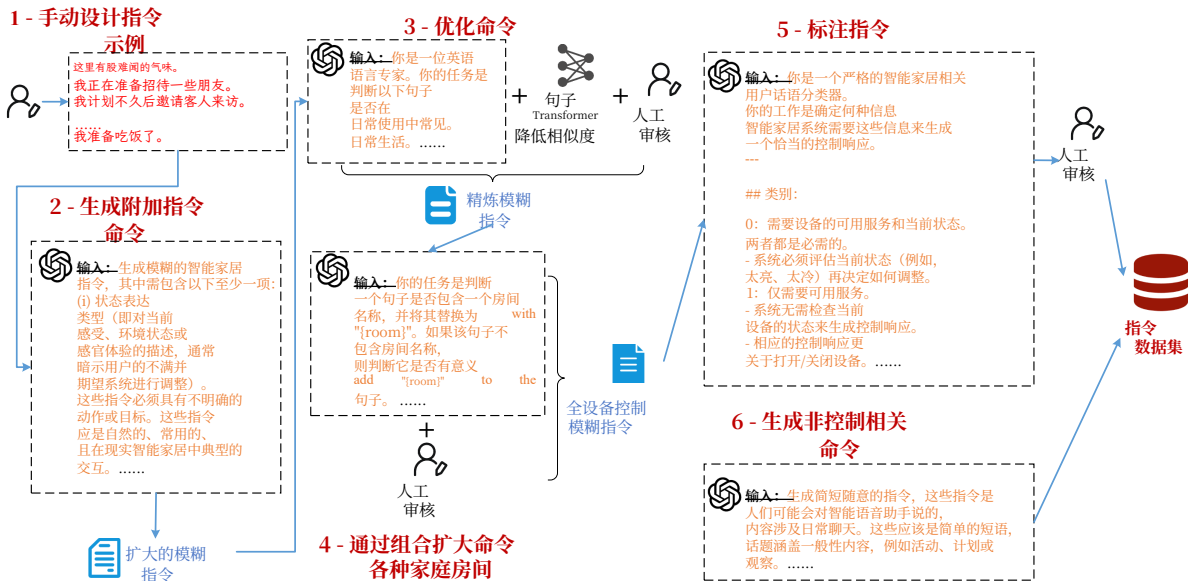


图10. 用于训练分类器的指令数据集C生成工作流程。

家用设备信息和已识别的控制因素，我们筛选相关设备的当前状态并生成一个受限输入提示。

(ii) 面向模糊指令理解的大语言模型微调。为了约束输出格式，并增强其在智能家居设备控制和模糊指令理解方面的能力与知识，我们使用生成的模糊指令控制数据集对一个轻量级大语言模型进行微调。在数据集生成过程中，我们使用一个高性能商用大语言模型来处理随机配对的模糊指令与设备信息组合，并生成相应的控制计划响应。为确保模糊指令控制数据集中生成的控制计划的准确性，我们从提供给高性能大语言模型的输入中过滤掉不相关的信息。

随后，将微调后的大语言模型部署在资源受限的边缘设备上，利用受限输入提示和定制的KV缓存技术，以低延迟生成合理的控制计划。

## 4 VCU-LLM的设计

### 4.1 模糊指令知识检索

算法1 概述 了模糊指令知识检索的过程，其目标是生成一个有限的设备状态列表  $O^*$  以及一个相关设备类型列表  $D^*$ ，以减少输入提示词元的数量。

st

4.1.1 所需信息识别。我们观察到，在智能家居设备控制场景中，用户的模糊指令可以根据所需设备信息的不同类型分为三类：

- (i) 设备服务和设备状态均为所需（例如，“我几乎听不见电视声音。”，这需要根据当前电视音量来调整音量）。
- (ii) 仅需设备服务（例如，“我想喝杯咖啡。”，它遵循固定的控制模式，并根据指令和家庭中所有可用服务来确定要激活的服务）。

**算法1: 模糊指令知识检索算法**


---

**输入:** 用户指令  $Q$ , 设备状态列表  $S = [s_1, s_2, s_3, \dots]$ , 句子转换器  $\mathbb{S}$ , 分类器  $\mathbb{C}$ , 词性标注模型  $\mathbb{P}$ , 模糊指令记忆库  $B$ , 关键词词典  $V$ , 控制因子-设备类型字典  $F$ 。

**输出:** 受限设备状态列表  $O^*$  和相关设备类型  $D^*$ 。

- 1 控制因子列表  $C^* \leftarrow \emptyset$ ;
- 2 受限设备状态列表  $O^* \leftarrow \emptyset$ ;
- 3 相关设备类型  $D^* \leftarrow \emptyset$ ;
- 4 词性标注项列表  $P \leftarrow \mathbb{P}(Q)$ ;
- 5  $cls$  词元嵌入  $E_{cls}$ , 平均池化嵌入  $E_{mean} \leftarrow \mathbb{S}(Q)$ ;
- 6 所需信息类型  $\leftarrow \mathbb{C}(E_{cls})$ ;
- 7 **如果** 所需信息类型 == 无需信息 **则**
- 8     返回 None;
- 9 **否则**
- 10   余弦相似度列表  $C \leftarrow$  计算  $E_{mean}$  与每个指令示例嵌入之间的余弦相似度于  $B$  中;
- 11   **如果** 任何  $c_i$  在  $C$  中大于阈值  $\tau$  **则**
- 12     获取在  $B$  中  $c_i$  最大的指令  $B_{oi}$ ;
- 13     从  $B$  中获取  $B_{oi}$  对应的控制因素  $B_{ci}$ ;
- 14     将  $B_{ci}$  追加到  $C^*$ ;
- 15   **对于**  $P$  中的每个词性标注项  $P_i$  **执行**
- 16     **如果**  $P_i$  是动词或  $P_i$  是动词短语 **则**
- 17       **如果**  $P_i$  在  $V$  中 **则**
- 18         从  $V$  中获取  $P_i$  对应的控制因素  $V_{ci}$ ;
- 19         将  $V_{ci}$  追加到  $C^*$ ;
- 20   **如果**  $C^*$  为空 **则**
- 21     返回 None;
- 22   **else**
- 23     针对  $C^*$  中的每个控制因素  $C_i^*$  **执行**
- 24       根据  $F$  和  $S$ , 获取  $C_i^*$  对应的设备状态和设备类型;
- 25       将对应设备的状态  $O_i^*$  追加到  $O^*$ ;
- 26       将对应设备类型  $D_i^*$  追加到  $D^*$ ;
- 27     **如果**  $D^*$  不为空 **则**
- 28       **如果** 所需信息类型 == 为仅需服务 **则**
- 29         返回  $D^*$ ;
- 30       **else**
- 31         返回  $O^*$  和  $D^*$ ;
- 32     **else**
- 33       返回 None;

---

(iii) 既不需要设备服务也不需要设备状态 (例如, “今天有什么新闻?”, 这与家庭环境中的任何变化无关)。

为降低设备端大语言模型的推理成本, 我们首先需要确定指令所需的信息类型。针对用户的指令  $Q$ , 我们使用一个类BERT的 [24, 47] 预训练轻量级句子转换器

表1. 控制因子-设备类型字典。

| 控制因子    | 对应设备类型          |
|---------|-----------------|
| 照明      | 灯、窗帘            |
| 温度      | 空调, 风扇, 窗帘      |
| 安防      | 门锁              |
| 湿度      | 加湿器             |
| 娱乐      | 电视、媒体播放器、扬声器    |
| 空气质量与气流 | 风扇、空气净化器、新风通风系统 |
| 家务      | 清洁机器人           |
| 其他      | 咖啡机             |

$\mathcal{S}$  [2] 进行编码。经过词元化后,  $Q$  中的每个词元被编码为一个384维的嵌入向量。我们提取与 [CLS] 词元对应的嵌入向量, 记为  $E_{cls}$ , 并将其输入一个三分类分类器  $\mathcal{C}$ , 以确定所需设备信息的类型 (即上述三种类型之一)。

为了训练该分类器  $\mathcal{C}$ , 我们构建了一个命令数据集  $\mathcal{C}$ , 其中包含各种指令及其对应的所需设备信息类型作为标签, 如图10所示。  $\mathcal{C}$  总共包含 4,895 个指令样本, 其中 2,392 个标记为类型 (i), 1,481 个标记为类型 (ii), 1,022 个标记为类型 (iii)。我们冻结句子转换器的参数以保留其特征提取能力, 并使用交叉熵损失来训练分类器参数。该分类器  $\mathcal{C}$  由两个全连接层组成。第一个全连接层有 128 个单元, 随后是层归一化 [20] 和 ReLU 激活函数。输出全连接层有 3 个单元, 没有层归一化或激活函数。命令数据集  $\mathcal{C}$  按 8:2 的比例划分为训练集和测试集。训练完成后, 该分类器在测试集上达到了 98.38% 的准确性。

如果输入指令被分类为类型 (iii), 则模糊指令知识检索过程终止。对于类型 (ii) 的指令, 在后续的设备端LLM推理过程中, 仅需在提示词中包含设备服务。与频繁变化的设备状态不同, 只要家用设备没有改变, 可用的设备服务就保持一致。对于类型 (iii) 的指令, 系统会直接返回“指令不可用”, 而无需调用LLM推理, 从而减少不必要的计算开销。

**4.1.2 控制因子识别与相关设备选择。** 为了识别类型 (i) 指令的相关设备状态以及类型 (ii) 指令的相关设备类型, 我们构建了一个控制因子-设备类型字典  $F$  和一个模糊命令记忆库  $B$ 。为了构建  $F$ , 我们首先识别出家庭中所有与智能设备 (家电设备) 相关的可用控制因素。然后, 基于现有商业智能家居系统中的智能家电类型 [1, 14, 15, 17], 我们列出常见的家用设备类型, 并为每个控制因子确定对应的设备类型。如表 1所示, 这种映射关系被存储为  $F$ 。模糊命令记忆库  $B$  存储了一小部分模糊指令示例的嵌入向量, 这些向量由句子转换器  $\mathcal{S}$ 提取, 并附有其对应的控制因素, 这些控制因素由一个大语言模型确定并经人工审核。记忆库中的指令示例是从  $B$  小节 4.1.1中的  $\mathcal{C}$  中手动选取的, 以确保内容的多样性和代表性。如算法1的第10-14行所示, 在获得输入指令  $Q$  的平均池化嵌入向量  $E_{mean}$  后, 我们计算其与  $B$  中  $E_{mean}$  个指令嵌入向量之间的余弦相似度。余弦相似度值超过阈值  $\tau$  且最高的对应控制因素将被添加到  $\mathcal{C}^*$ 中。为了平衡计算复杂度和控制因素投影的准确性, 我们最终选取了119个指令示例来构成  $B$ 。

然而, 仅依靠  $B$  中的指令示例不足以准确确定所有输入指令对应的控制因素。因此, 我们进一步利用一个词性标注分析模型[31] 来辅助补充控制因素, 如算法1第15-19行所示。该轻量级词性分析模型根据上下文分离句子中的单词和短语, 并为每个项目分配适当的标签 (例如名词、动词、形容词)。我们发现某些词语与特定的

控制因素存在明确的对应关系（例如，动词“加热”对应温度）。因此，我们构建了一个关键动词词典  $V$ ，其中包含一组关键词及其关联的控制因素。该词典共包含144个关键词。对于句子中的每个项目，我们检查其是否存在于  $V$  中；如果存在，则将其关联的控制因素追加到  $C^*$ 。

为输入指令  $Q$  获取对应的控制因子列表  $C^*$  后，下一步是识别与每个控制因子关联的设备。如果  $C^*$  在第19行后仍为空，则该输入指令被视为无效指令，不会调用LLM推理。否则，将根据  $F$ 、 $S$  和  $C^*$  检索相关设备，形成受限设备状态列表  $O^*$  和相关设备  $D^*$ 。对于  $C^*$  中的每个控制因子  $C_i^*$ ，如果在  $S$  中找不到对应的设备类型，则  $O_i^*$  和  $D_i^*$  均设为 None。处理完所有  $C_i^*$  后，如果  $D^*$  仍为空，则表明基于当前家庭中可用的设备类型无法执行该指令。识别相关设备可以显著缩短输入提示的长度。此外，拥有明确界定的设备范围有助于缓解大语言模型中的幻觉问题，而受限的上下文能让大语言模型专注于控制相关设备，从而生成更合理的指令。

先前的工作ChatIoT [26], 也实现了一种结合句子转换器的压缩提示技术，以过滤掉不相关信息。然而，ChatIoT的提示压缩仅限于过滤相关设备和服务。相比之下，在模糊指令知识检索中，我们首先判断用户的指令是否需要设备状态。我们并非直接从所有可用设备中识别相关设备，而是先选择相关的控制因素，这些因素范围更有限且更容易从用户意图中识别，然后将这些因素映射到对应的设备。因此，最终的受限输入提示包含更少的不相关信息，相关设备的识别也更为准确。

## 4.2 面向模糊指令理解的大语言模型微调

在获得受限提示后，我们使用微调后的大语言模型作为智能家居助手，在边缘设备本地生成合理的控制计划。在本小节中，我们首先定义用于设备端LLM推理的输入提示结构。接着，我们描述用于指导微调大语言模型的模糊指令控制数据集的生成策略，以及微调过程本身。最后，我们描述了使用微调后的大语言模型进行设备端推理的优化过程。

**4.2.1 输入提示结构。** 为提高设备端推理效率，输入提示必须具有精简的结构。如图11所示，我们用于设备端LLM推理的输入提示由五个部分组成。(i) **固定系统提示**。它在所有家庭和指令中保持不变。由于LLM将使用任务相关数据进行微调，系统提示无需包含详细说明，从而减少了其长度。(ii) **家庭中所有设备的服务信息**。关于人体传感器（用于检测附近人员存在）的信息不包含在提示的此部分，因为它是只读设备。当家庭中的设备没有变化时，提示此部分的内容保持不变。(iii) **与用户指令相关的设备类型**。模糊指令知识检索算法的输出之一。当相关设备的状态为None时，提示的此部分帮助LLM根据家庭中所有可用服务（如第(i)节所列）确定控制计划。(iv) **与用户指令相关的设备的当前状态信息**。模糊指令知识检索算法的输出之一。如果用户指令被分类为“仅需服务”，则提示的此部分可设置为“None”。所有人体传感器的状态均包含在内。与提供所有房间中所有设备的当前状态相比，这显著缩短了提示的长度。(v) **用户指令**。用户对其希望改变或不满意的家庭内因素的自然表达。

基于包含上述五个组件的输入提示，助手会生成一个具体的控制计划。如图11所示，输出控制计划的每一行都包含设备类型、其所在房间信息以及

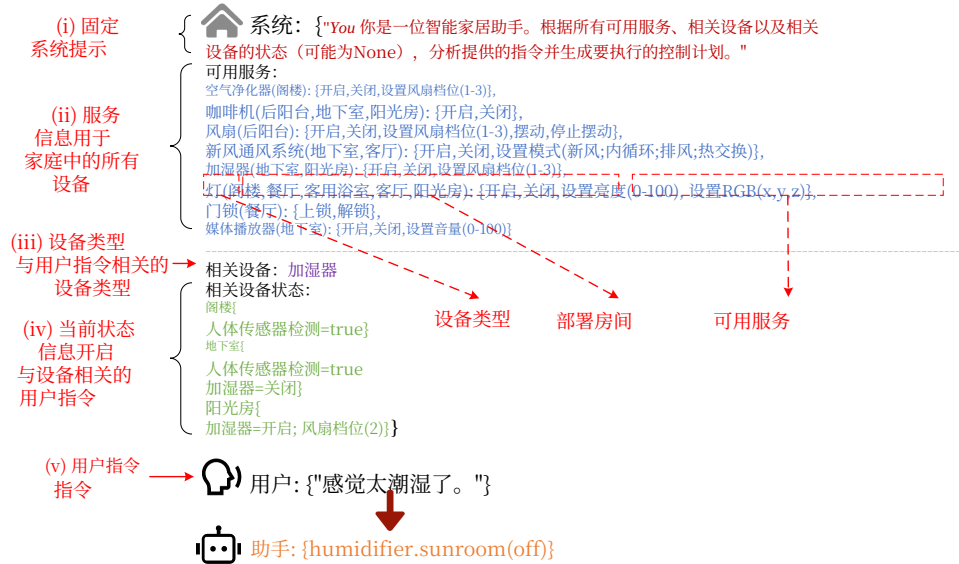


图11. 设备端LLM推理的输入提示结构与内容，以及智能家居助手的输出。

要更改的服务，全部以JSON格式呈现。如果当前设备状态无法进一步优化以满足用户指令中表达的要求（例如，房间内灯的亮度已经达到100%，但用户指令是“这里太暗了”），则输出控制计划可能为“None”。此外，与控制无关的指令以及当前家庭中设备类型无法满足的指令，已在模糊指令知识检索过程中被过滤掉。

**4.2.2 模糊指令控制数据集生成。**为生成数据集，我们首先创建一个仅包含相关设备信息、不含控制计划响应的模糊指令控制数据集 ( $\mathcal{D}_u$ )。如算法2所示，为了纳入多种设备类型、房间布局和设备状态，同时确保每个生成的样本至少包含一种与用户指令中控制因素相关的设备类型，我们为每条指令生成五种不同类型的样本。这些变体基于指令是否包含明确的房间名称、是否包含所有相关设备类型、以及是否覆盖人体传感器等。输入  $V_1$  和  $V_2$  来自第4.1.1小节中的命令数据集  $\mathcal{C}$ （类型(i)和类型(ii)）。

为获取控制计划响应，我们将数据集  $\mathcal{D}_u$  中的样本输入到一个高性能大语言模型  $M$  [61] 中，并使用一个精心设计、包含示例的详细提示来指导生成相应的控制计划。值得注意的是，数据集  $\mathcal{D}_u$  仅包含与输入指令相关的设备服务和状态。通过排除与指令无关的设备信息的影响，由  $M$  生成的控制计划响应是准确且合理的。在获得控制计划响应后，为生成未经过滤的模糊指令控制数据集  $\mathcal{D}$ ，我们会随机添加与指令无关的设备，并为它们分配随机的当前状态，以确保数据多样性并模拟家用设备的真实使用场景。

在最后步骤中，我们根据设备端LLM推理的输入提示结构（如图11所示）组织  $\mathcal{D}$  中的每个样本，并应用模糊指令知识检索算法进行过滤，以获得用于大语言模型微调的最终训练数据集  $\mathcal{D}_t$ 。具体来说，我们根据模糊指令知识检索的结果过滤相关设备类型及其

**算法2:** 生成未过滤模糊指令控制数据集的算法

---

**输入:**  $V_1$ : 带有指定控制因素的模糊指令;  
 $V_2$ : 带有{room}占位符和指定控制因素的模糊指令;  
 $F$ : 控制因素-设备类型字典;  
 $RS$ : 所有房间的集合;  
 $DS$ : 所有设备的集合;  
 $M$ : 高性能大语言模型;

**输出:** 未过滤模糊指令控制数据集  $\leftarrow D$ ;

- 1 仅含相关设备信息且不含控制计划响应的模糊指令控制数据集  $\leftarrow Du$ ;
- 2 未过滤模糊指令控制数据集  $\leftarrow D$ ;
- 3 **对于** 每条指令  $Q$  于  $V_1 \cup V_2$  **执行**
- 4     **对于**  $i = 1$  至  $N$  **执行**
- 5         查找  $Q$  对应的目标设备集  $\leftarrow TDS$ ;
- 6         生成一个随机子集  $RS_0 \subseteq RS$ ;
- 7         将  $RS_0$  随机分割为两组  $G_1$  与  $G_2$ ;
- 8         **若**  $Q \in V_1$  **则**
- 9             生成未过滤样本  $R_A$ , 条件如下:
  - 在  $RS_0$  中实现完整人体传感器覆盖;
  - $G_1$  获取  $TDS$  中的所有设备, 而  $G_2$  仅获取  $TDS$  的一个随机子集;
 生成未过滤样本  $R_B$ , 条件如下:
  - 在  $RS_0$  中实现随机人体传感器覆盖;
  - $G_1$  获取  $TDS$  中的所有设备, 而  $G_2$  仅获取  $TDS$  中的部分设备;
 生成未过滤样本  $R_C$ , 要求:
  - 在  $RS_0$  中随机布置人体传感器覆盖;
  - 确保来自  $TDS$  的至少一个设备部署在  $RS_0$  的至少一个房间内, 且同时满足  $G_1$  和  $G_2$ ;
- 10         **else**
- 11             生成未过滤样本  $R_D$ , 要求:
  - 在  $RS_0$  中随机布置人体传感器覆盖;
  - 来自  $TDS$  的至少一个设备部署在  $RS_0$  的至少一个房间内;
  - 将  $Q$  中的{room}占位符替换为  $RS_0$  中的一个随机房间;
- 12         为  $R_i$  中的每个设备随机分配一个当前状态;
- 13         将生成的样本  $R_i$  添加到  $Du$ ;
- 14 **对于** 数据集  $\langle D \rangle_u$  中的每个样本, **执行**
- 15     利用模型  $M$  获取  $R_i$  的控制计划响应;
- 16     将控制计划响应追加到  $R_i$ ;
- 17     将随机无关设备  $DS \setminus TDS$  添加到现有房间  $RS_0$  中  $R_i$ ;
- 18     为  $R_i$  中的每个无关设备随机分配一个当前状态 (如果已添加);
- 19     将  $R_i$  添加到  $D$ ;
- 20 **返回**  $D$ ;

---

状态, 同时保留服务部分中的所有可用设备。

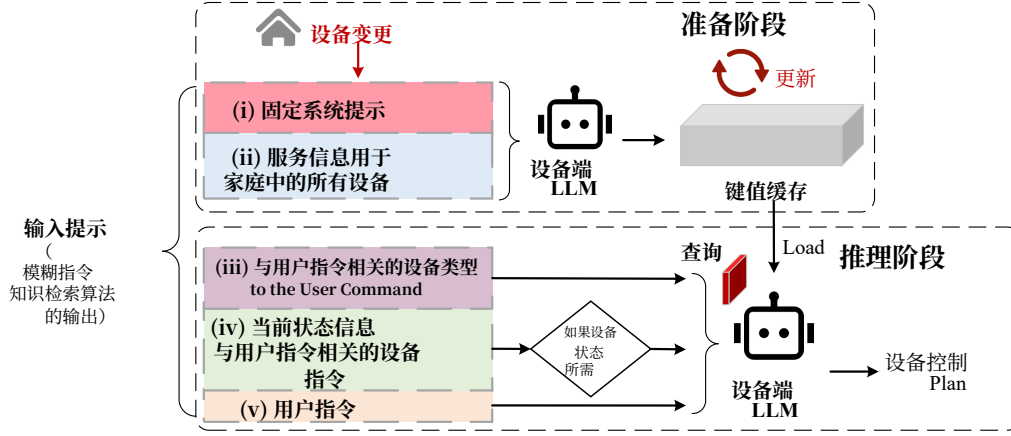


图 12. LLM设备端推理的工作流程。

4.2.3 大语言模型微调。我们利用包含超过40,000个样本的模糊指令控制数据集 $D_t$ ，通过指令微调 [39] 对预训练大语言模型进行微调。在微调过程中，一个具有参数  $\theta$  的预训练LLM会使用模糊指令数据集  $D_t$  进行适配，该数据集由指令-响应对  $(x^{(i)} y^{(i)})_{i=1}^N$  组成。其中，  $x^{(i)}$  代表一个输入提示，对应于图11中的 (i)至(v)，而  $y^{(i)} = (y^{(i)}_1, y^{(i)}_2, \dots, y^{(i)}_{T_i})$  表示长度为  $T_i$  的目标令牌序列，代表最优控制计划。该模型将令牌上的条件概率分布定义为：

$$p_{\theta}(y^{(i)} | x^{(i)}) = \prod_{t=1}^{T_i} p_{\theta}(y_t^{(i)} | x^{(i)}, y_{<t}^{(i)}),$$

其中  $y_{<t}^{(i)} = (y^{(i)}_1, \dots, y^{(i)}_{t-1})$  代表前序令牌。微调目标是最小化数据集 $D_t$ 中所有样本的负对数似然损失：

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{t=1}^{T_i} \log p_{\theta}(y_t^{(i)} | x^{(i)}, y_{<t}^{(i)}),$$

其中  $\theta$  基于  $\nabla_{\theta} \mathcal{L}(\theta)$  使用梯度下降法进行优化。

我们还测试了微调过程中低秩适应 (LoRA) [32] 的性能。在这种情况下，原始模型权重  $\theta$  被冻结，并通过可训练矩阵  $A \in \mathbb{R}^{d \times r}$  和  $B \in \mathbb{R}^{r \times d}$  引入低秩分解，其中  $r \ll d$ 。适配权重为： $\theta_0 = \theta + BA$ 。损失函数  $\mathcal{L}(\theta_0)$  保持不变，但仅更新矩阵  $A$  和  $B$ ： $\nabla_{A,B} \mathcal{L}(\theta_0 + BA)$ 。该技术显著减少了可训练参数的数量，同时保持了原始的优化目标。然而，尽管LoRA降低了训练成本并带来更稳定的损失收敛，但它可能会限制微调后的大语言模型的表达能力。

在微调过程中，我们还引入了一个正则化项，以惩罚与预训练LLM的偏差，有助于缓解过拟合。对于标准微调和基于LoRA的微调，损失计算均采用标签掩码，其中非目标令牌（例如，输入提示）通过将其标签设置为 -100来进行掩码。这确保了只有LLM生成的响应令牌对损失 $\mathcal{L}(\theta)$ 有贡献。

**4.2.4 基于KV缓存优化的设备端推理。**在获得微调模型  $\theta^*$  后, 我们根据任务特性优化其设备端推理。如 4.2.1 小节所述, 如果设备没有变化, 那么部分 (i) 和 (ii) 对每个家庭是固定不变的。因此, 一旦 LLM 部署在本地设备上并获取了该家庭的设备信息, 我们便应用 KV 缓存 [42, 62, 63] 来加速后续的 LLM 推理。在大语言模型的自回归解码过程中, 在每个时间步  $t$ , 模型会对所有过去的令牌  $1, \dots, t$  计算自注意力。这种朴素方法会导致序列长度的二次方成本。为了降低此成本, KV 缓存存储了先前步骤中计算出的键 (K) 和值 (V) 投影, 允许复用并减少计算。对于解码器层  $l$ , 记第  $t$  个令牌的隐藏状态为  $h_t^l$ 。其投影为

$$k_t^l = h_t^l W_K^l, \quad v_t^l = h_t^l W_V^l,$$

并且缓存通过以下方式更新

$$K_{1:t}^l = \text{concat}(K_{1:t-1}^l, k_t^l), \quad V_{1:t}^l = \text{concat}(V_{1:t-1}^l, v_t^l).$$

在下一个词元生成期间, 仅计算新的查询  $q_{t+1}^l = h_{t+1}^l W_Q^l$ , 注意力由以下方式给出

$$\text{Attention}(q_{t+1}^l, K_{1:t}^l, V_{1:t}^l) = \text{softmax}\left(\frac{q_{t+1}^l K_{1:t}^{l\top}}{\sqrt{d}}\right) V_{1:t}^l.$$

这减少了重复计算  $K$ ,  $V$  适用于所有 past 令牌和  $\text{channel complexity}$  来自  $O(t^2)$  到  $O(t)$  per step.

尽管 KV 缓存是设备端 LLM 推理的一项基础优化技术, 但 VCU-LLM 重组了输入提示的结构和内容, 以最大化其效果。我们以最简洁的形式呈现 LLM 生成合理控制方案所需的所有信息。为了实现不同模糊指令之间的高效切换, 并最小化设备端推理开销, 我们将智能家电信息分离为静态属性 (服务信息) 和动态属性 (当前状态), 确保提示的大部分内容已预填充在 KV 缓存中。

如图 12 所示, 在将 LLM 部署到家庭边缘设备后, 我们首先在准备阶段将提示的第 (i) 和 (ii) 部分输入到设备端大语言模型中, 以获得相应的 KV 缓存, 并将其存储在内存中。如果家庭中的设备发生变化, 我们会更新相关的令牌并重新计算对应的 KV 缓存。在推理阶段, 我们首先从内存中加载先前存储的 KV 缓存, 并判断是否需要设备状态信息。如果需要, 则将第 (iii) 和 (iv) 部分连同用户指令作为新令牌输入到设备端大语言模型中。否则, 仅将用户指令作为新令牌提供。如图 11 所示, 第 (i) 和 (ii) 部分通常占据了提示中的大部分令牌。同时, 在推理阶段, 新令牌会经过模糊指令知识检索算法的筛选, 进一步降低了计算需求。

## 5 评估

### 5.1 实验设置

**5.1.1 基线模型与评估指标。** **基线模型:** 我们根据参数量将基线模型分为三个层级。大规模模型包括 GPT-4o 和 GPT-3.5 [18]。中等规模模型包括 Llama3-8B [25] 和 Llama-7B [55]。小规模模型包括 HomeLLM [6] 和 Llama3.2-1B [25]。其中, HomeLLM 是通过在智能家居设备控制数据集 [5] 上对 TinyLlama-1.1B [65] 进行微调得到的。该数据集仅包含显式用户指令, 并根据预定义规则生成相应的控制计划。对于所有在边缘设备上运行的小规模模型 (包括 VCU-LLM), 我们应用 INT8 量化 [3] 以降低推理延迟。我们在 VCU-LLM 中使用的模型是基于 Llama3.2-1B 微调得到的。

**基线推理策略:** 与第 2 节类似, 我们采用两种大语言模型推理方法: 单次推理和迭代推理。对于小规模和中等规模模型, 考虑到模型能力, 仅应用单次推理。

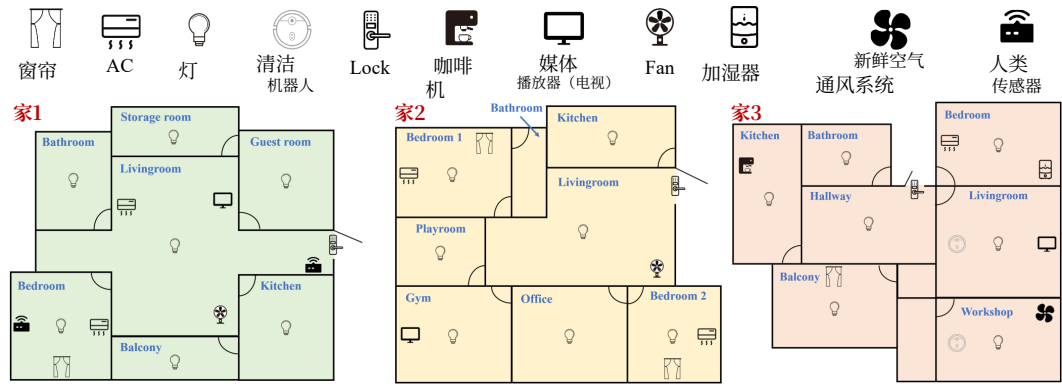


图13. 图 用于基于人类的控制计划评分的三个家庭模板。评分。

对于除HomeLLM外的所有基线模型和推理策略组合，我们使用完整的提示内容（如图1所示）作为模型输入。对于HomeLLM，我们首先将设备信息转换为其微调数据集的格式，然后将其输入微调模型，以确保其能力得到充分利用。在推理过程中，我们将所有模型的温度设置为较低值（0.3），以确保输出更稳定。

**评估指标：**为了全面评估生成的控制计划的质量，我们结合LLM评分和人工评估，对由不同模型与推理策略组合生成的控制计划进行评分：

- 对于LLM评分，我们指导另一个强大的大语言模型（DeepSeek-R1-0528 [29]，该模型与第4节中使用的所有基线模型和大语言模型均不同）以1到5分的尺度，从三个方面对每个控制计划进行评分：简洁性与可执行性（记为**简洁性**）、所涉及设备的准确性（记为**准确性**），以及最终状态变化的适宜性（记为**适宜性**）。最终分数是这三个评分的平均值。我们使用随机构建的、不同于模糊指令控制数据集 $\mathcal{D}$ 的设备信息生成了500个样本，并要求不同的基线大语言模型和推理策略基于此信息生成控制计划。在评分过程中，设备信息（输入）和相应的控制计划（输出）会一并提供给评分大语言模型。值得注意的是，为了公平比较并充分评估我们的模糊指令知识检索算法的有效性，用于LLM评分的设备信息包含了所有基线模型以及我们的VCU-LLM所使用的完整未过滤信息。
- 对于人工评分，类似于第2节，我们要求参与者通过问卷，对智能家居助手提供的控制计划在1（非常不满意）到5（非常满意）的范围内评价“满意度”。为了降低参与者在评估生成的控制计划时的认知负荷，并提高评分可靠性，我们设计了三种具有不同设备类型及其当前状态的常见家居布局，如图13所示。每个家庭模板包含六条模糊指令。问卷设计与第2.1.2小节类似，包括家庭模板图示、每个设备的可用服务及其当前状态。评分时，除了展示不同模型生成的原始内容外，我们还为每个输出提供了一份简洁的人工摘要，重点关注设备状态的变化，以帮助用户理解模型输出并减轻参与者的工作量。我们在问卷中说明，这些摘要仅供参考，不属于评分标准。

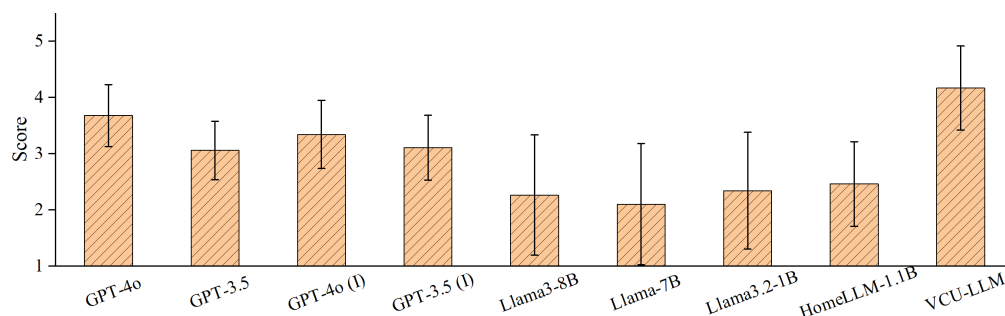


图 14. 基线和我们 VCU-LLM 的人工评分结果。误差棒代表不同参与者计算得出的标准差。

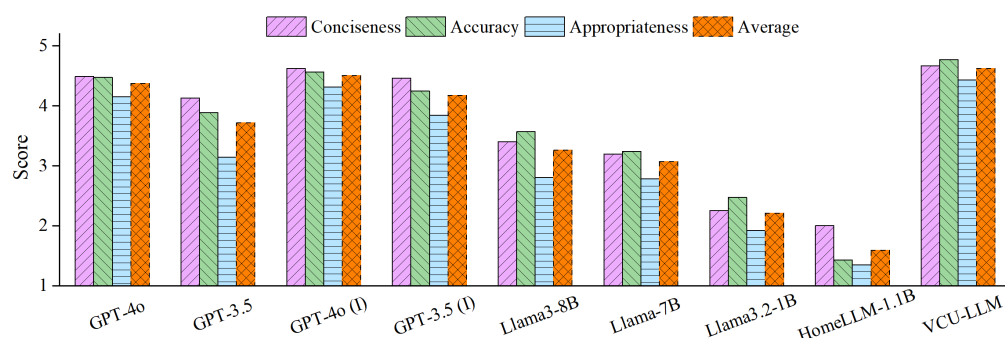


图15. 基线模型与我们VCU-LLM的LLM评分结果。

对于第 5.4 和 5.5 节中，仅应用了 LLM 评分，因为验证不同模块有效性所需的大量实验会给参与者带来过重负担。

**5.1.2 招募的参与者。** 我们通过电子邮件、社交媒体和口头沟通招募了 36 名参与者进行人工评分。其中，20 人年龄在 18-25 岁，5 人 26-30 岁，3 人 31-40 岁，6 人 41-50 岁，2 人 51-60 岁。性别方面，7 人为女性，29 人为男性。在问卷填写期间，参与者不得相互交流，且以匿名方式收集回答。每位参与者需完成 171 项评分任务，整个过程约需 40 分钟。在所有参与者中，有 8 人被选中参与 5.6 节所述的案例研究实验。

**5.1.3 硬件。** 实验中使用了云服务器和边缘设备。云服务器配备英特尔至强铂金 8375C @ 2.90GHz CPU 和 2× 英伟达 A100 GPU。中等规模模型的推理和指令微调过程在云服务器上执行。使用的边缘设备是树莓派 5 [12]，这是在家庭环境中运行 AI 模型广泛采用的终端，配备 2.4GHz Arm Cortex-A76 CPU，价格低于 80 美元。包括 VCU-LLM 在内的小规模基线模型的推理过程在树莓派 5 上进行。

## 5.2 生成控制计划的质量

如图 14 和图 15 所示，VCU-LLM 在生成的控制计划质量上，无论是人工评分还是 LLM 评分，均展现出显著提升，平均得分分别高出 37.5% 和 49.1%。

此外，我们观察到，在人工评分和LLM评分中，大规模基线模型的得分均高于中等规模和小规模基线模型，平均增益分别为13.4%和65.4%。在LLM评分中，HomeLLM-1.1B得分最低，因为它是专门为处理显式指令而微调的。然而，在人工评分中，HomeLLM-1.1B在中等规模和小规模基线模型中获得了最高分。这可能是因为Llama3-8B、Llama-7B和Llama3.2-1B产生了过于冗余的输出，导致参与者失去耐心，未能完整阅读所有相关控制信息。根据图14中的误差条显示，GPT-4o在整体上表现出最稳定的人工评分，而在中等规模和小规模方法中，我们的VCU-LLM则展现出最稳定的性能。

为了进一步揭示模糊指令理解的系统性局限和系统瓶颈，我们结合了基线方法和VCU-LLM在人工评分问卷中报告的参与者不满原因，提供了一项细粒度的失败分析。我们识别出以下7种常见的失败类型：

- **该模糊指令被认为是不可实现或与家电控制无关。**诸如“帮我省点电”或“我很快要起床了”之类的命令，通常被大规模大语言模型基线认为与家电控制无关，并且当应用迭代推理时，这个问题变得更加严重。这可能是因为迭代推理要求模型在获取可用家电设备信息之前，首先判断模糊指令是否可执行，这导致了上下文信息不足。相比之下，VCU-LLM采用了一个在命令数据集上训练的专用轻量级分类器，来识别指令是否与控制相关。
- **控制计划中选择了无关设备。**这种情况在所有基线方法中都会发生，我们将其归因于大语言模型对模糊指令的理解不足，以及不同家电设备功能的多样性。例如，对于指令“给我带点新鲜空气”，基线方法有时会选择打开窗帘，而不是启动新风通风系统。相比之下，我们的VCU-LLM通过模糊指令知识检索，能够利用控制因素来准确识别相关设备。
- **控制计划中选择了不存在的设备。**这主要发生在中等规模和小规模LLM基线中，它们表现出幻觉问题。该问题在HomeLLM中更为明显，可能是因为它是在一个由显式指令组成的数据集上进行微调的，这使其更容易生成带有固有偏见的输出。通过在定制的模糊指令控制数据集上进行微调，VCU-LLM可以有效减少幻觉问题。
- **控制计划中涉及错误的设备状态变更。**例如，一些生成的控制计划使用开尔文值而非RGB值来调整灯光颜色温度。这发生在中等规模和小规模LLM基线中，这是由于它们对家电服务信息的理解不足。VCU-LLM中的大语言模型微调使用指定的服务标准化了控制计划的格式。
- **未能生成控制相关输出。**这发生在中等规模和小规模LLM基线中，针对某些更难以与特定设备控制关联的模糊指令（例如，“我们来开个派对吧。”）。这些基线有时在其输出中仅仅是重复输入的服务信息和状态。通过将来自模糊指令知识检索的限定提示词与在定制化模糊指令控制数据集上的微调相结合，VCU-LLM可以完全避免此问题。
- **控制计划中设备状态变更的具体参数无法满足所有参与者。**这是因为，即使对于同一模糊指令，不同用户希望达成的控制结果在细粒度方式上也可能存在差异（例如，媒体播放器的音量、灯的亮度）。这对于包括我们的VCU-LLM在内的所有方法都是一个常见问题。

- **具有强主观性的指令，其控制计划无法满足所有参与者。**例如，对于“我的朋友要离开了”这条指令，有些用户期望解锁门锁，而另一些用户则期望关闭媒体播放器。这对于包括我们的VCU-LLM在内的所有方法都是一个常见问题。

### 5.3 系统开销

我们首先通过平均与图13中三个家庭模板相关的模糊指令的延迟，来测试不同基线方法和VCU-LLM的延迟。每个控制计划生成三次以获得最终的平均延迟。对于在边缘设备上运行的方法，我们忽略了VCU-LLM中的模型加载时间和KV缓存初始化时间，因为这些开销是部署到家庭后的一次性成本。如图16所示，VCU-LLM的平均延迟为5.47秒，其中大部分时间消耗在LLM推理过程上。模糊指令知识检索算法仅耗时94.38毫秒。与在边缘设备上运行推理的小规模模型相比，VCU-LLM平均减少了8.44×的时间开销。与使用受限输入提示和输出格式的HomeLLM-1.1B相比，VCU-LLM在保持相似模型参数规模的同时，将开销降低了2.85×。与在配备A100 GPU的服务器上进行中等规模模型推理相比，VCU-LLM平均减少了3.44×的时间开销。使用基于单次API推理的大规模模型实现了最低的延迟，平均为2.89秒，而迭代推理则将延迟平均增加到11.83秒。我们还评估了不使用KV缓存技术的VCU-LLM的延迟（图16中的VCU-LLM (w/o)）。结果表明，KV缓存有效地将三个家庭模板的平均时间开销减少了5.67秒。为了更好地理解VCU-LLM在不同模糊指令和家居布局下的延迟分布，我们进行了额外的分析。如图17所示，家庭3的输入令牌数量最多，因为它包含最多种类的家电设备，有九种不同的设备类型。输入提示长度也受模糊指令的影响。例如，在家庭3中，处理指令“我想节省一些能源”时，大多数家电设备的当前状态都包含在输入提示中。在3个家庭模板的所有18条模糊指令中，延迟最长的是指令“我想放松一下”，为13.54秒，该指令生成了157个输出令牌，并控制了七种类型的家电设备，包括不同房间的灯、空调、风扇、媒体播放器和窗帘。相比之下，延迟最短的是指令“让客人进来”，仅需1.91秒，该指令只解锁门锁。总体而言，控制计划生成的延迟主要取决于满足模糊指令需要控制的家电设备数量，以及家庭中可用的家电设备。

我们随后在边缘设备上测试了小规模基线模型和VCU-LLM的内存使用情况。如图18所示，在整个过程中，VCU-LLM占用的内存少于1.5 GB，这非常轻量级，并为边缘设备上运行的其他并发进程留出了空间。Llama3.2-1B消耗更多内存，且其内存使用量随时间增加。这是因为该模型未经微调，导致产生大量包含冗余信息的输出令牌。同时，我们观察到KV缓存的使用对VCU-LLM的峰值内存使用量没有显著影响。

### 5.4 模糊指令知识检索算法的有效性

为证明模糊指令知识检索算法的有效性，我们将其过滤后的输出作为基线模型的输入提示，同时保持家用设备信息不变。GPT-4o和具有迭代推理功能的GPT-3.5被排除在外，因为其迭代推理过程的核心思想同样涉及基于指令相关性过滤设备。如图19所示，未经微调时，使用模糊指令知识检索算法生成的设备控制计划在所有基线模型上均获得了更高的分数，平均提升了12.1%。更重要的是，借助模糊指令知识检索算法，输入提示被显著缩减，平均实现了4.46的令牌减少率，

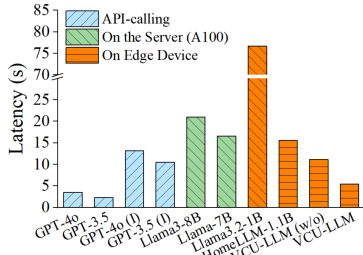


图16. 不同基线方法与VCU-LLM的平均时间开销。“w/o”表示未使用KV缓存技术。

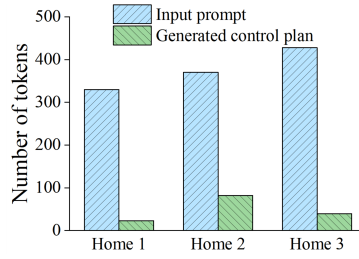


图17. VCU-LLM在不同家庭模板间的词元分布。

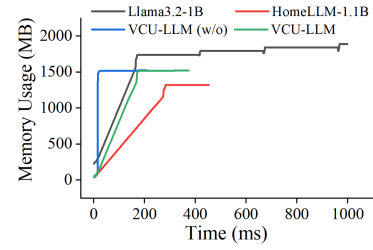


图18. 小规模基线方法与VCU-LLM在边缘设备上的内存使用情况。“w/o”表示未使用KV缓存技术。

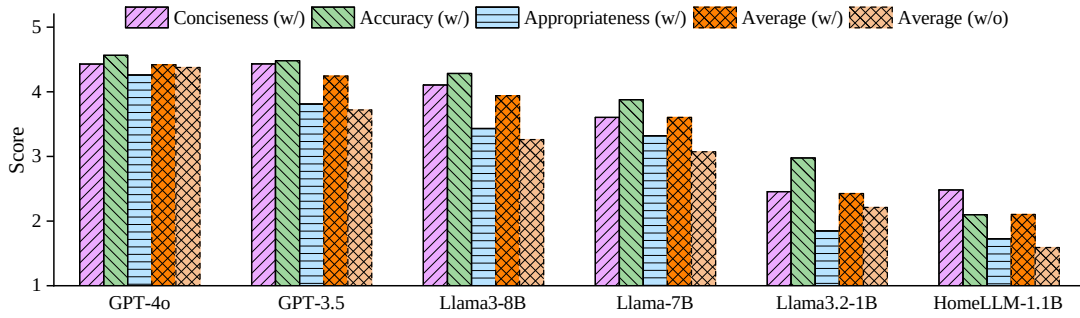


图19. 使用与不使用模糊指令知识检索的LLM评分对比val算法。

这极大地降低了大语言模型所需的推理时间。我们还使用原始的未过滤模糊指令控制数据集 $\mathcal{D}$ 和过滤数据集 $\mathcal{D}_f$ 对Llama3.2-1B进行了微调。在测试期间，数据格式与训练数据保持一致。如图20所示，对于使用过滤数据进行微调 and 测试的模型，其生成的控制计划分数提升了10.8%。此外，利用模糊指令知识检索算法生成的过滤数据，输出控制计划中所涉及设备的准确性提高了11.7%。

为检验模糊指令知识检索算法的召回率，我们从原始未过滤模糊指令控制数据集中随机抽取3,000个样本 $\mathcal{D}$ 。随后，我们使用另一大语言模型（DeepSeek v3.2）配合详细提示词来识别相关设备。模糊指令知识检索算法生成的相关设备列表与该大语言模型识别结果之间的召回率为0.94。我们检查了不匹配的案例，发现大多数错误是由大语言模型引入的。例如，大语言模型可能对模糊指令表现出刻板解释，倾向于调整家中状态变化显著的设备，或对已满足用户指令的设备状态不敏感。在大多数情况下，模糊指令知识检索算法生成的相关设备是合适的。然而，对于一些更具挑战性的模糊指令（例如“我想为一天做准备”或“我想提高效率”），检索到的设备集有时可能不够全面。如图22所示，我们评估了微调后的大语言模型在不同输入设置下的控制计划质量。“w/”表示使用所有可用设备且不过滤，“w/ compress text”表示使用ChatIoT中的过滤算法[26]，“w/ LLM”表示使用

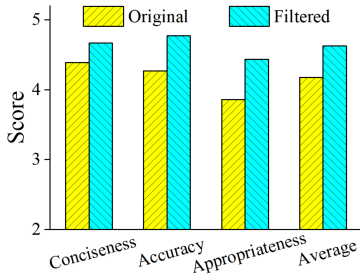
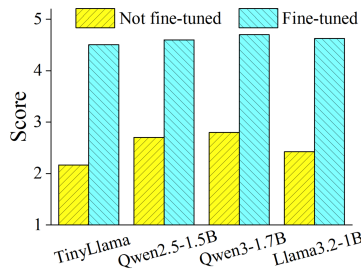
图 20. 使用过滤后数据集  $D$  进行微调的效果。

图 21. 大语言模型微调的效果。

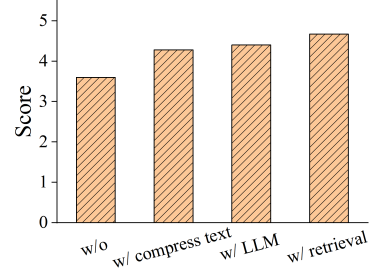


图 22. 模糊指令知识检索的效果。

大语言模型过滤后的设备，而“w/ retrieval”表示使用模糊指令知识检索过滤后的设备。“w/ compress text”通过从预训练Transformer模型中提取词嵌入并应用相应的聚类算法来过滤相关设备。“w/retrieval”下的控制计划质量比“w/ LLM”下高5.9%，比“w/ compress text”下高9.1%。我们还观察到，即使在不经筛选提供所有可用设备的情况下，微调后的大语言模型有时也能正确控制相关设备，且质量有所提升。

### 5.5 Effe使用定制化模糊指令控制数据集进行大语言模型微调的效果

为验证使用定制化模糊指令数据集进行微调的有效性，我们评估了由TinyLlama、Qwen2.5-1.5B [53]，Qwen3-1.7B [61]，和Llama3.2-1B在是否使用 $D_t$ 进行微调的情况下所生成控制计划的得分。测试期间，数据格式与筛选后的训练数据保持一致。如图21所示，对于所有预训练大语言模型，使用 $D_t$ 进行微调显著提升了生成的设备控制计划的质量，平均提升幅度达83.0%。Qwen3-1.7B的得分略高于Llama3.2-1B。然而，考虑到更大参数量带来的额外计算开销，我们在系统中采用了Llama3.2-1B作为本地大语言模型。

若未在定制化模糊指令数据集上进行微调，轻量化大语言模型会表现出以下问题，导致其性能得分下降：

- **输出中包含了无关设备。**即使通过模糊指令知识检索明确了相关设备，并将其呈现在输入提示中，轻量化大语言模型仍然会从服务信息中生成与当前用户指令无关的设备。
- **对设备状态的更改常常不合理。**对特定设备状态（如温度、风扇档位或灯光颜色）的修改不符合用户要求，或超出了正确范围。例如，当用户说“让灯光颜色更暖一些”时，轻量化大语言模型总是将RGB值设置为(255, 255, 255)，导致灯光反而变成了白色。
- **输出的控制指令格式不正确。**生成的控制计划有时是纯文本而非JSON格式。即使是JSON格式，也常常不严格遵循服务信息中的定义，导致无法将控制计划转换为可执行消息，从而使家庭设备无法控制。

通过在系统提示中提供严格详细的指令或额外示例，可以缓解上述部分不足，但这不可避免地会导致高昂的本地推理成本。因此，对于能力有限的轻量化大语言模型，有必要使用定制数据集进行监督微调，以提升智能家居中的模糊指令理解和设备控制能力，因为仅靠模糊指令知识检索无法实现这一点。

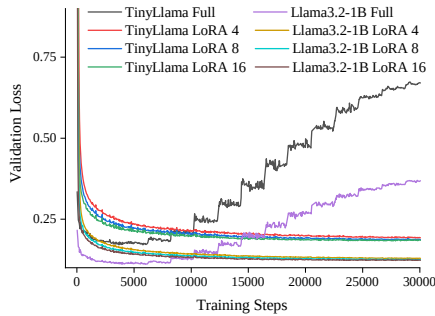


图 23. VCU-LLM的收敛性能。

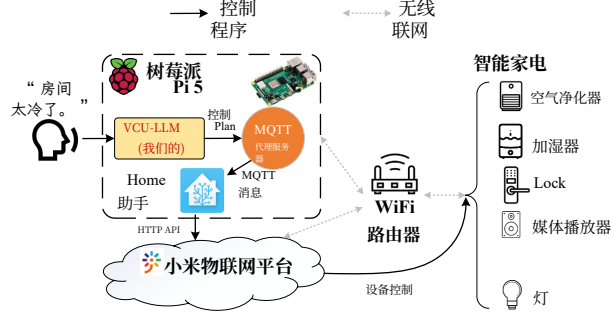


图 24. 在实际场景中部署VCU-LLM的框架。



图25. 案例研究的家庭环境。

在模型微调过程中，我们使用较小的学习率（ $2e-5$ ），以AdamW作为优化器，并应用余弦学习率衰减。批次大小设置为16，上下文长度设置为2048个令牌。微调数据集 $\mathcal{D}_t$ 包含41,012个问答对，其中80%用作训练集，20%用作验证集以评估微调效果。如图23所示，我们同时应用了全参数微调 and 不同秩的LoRA。与LoRA相比，全参数微调收敛更快，但随着训练步数增加，容易过拟合。使用LoRA时，更高的秩会带来更低的最终验证损失。我们选择验证损失最低的Llama3.2-1B全参数微调模型作为VCU-LLM中的微调后的大语言模型。

### 5.6 真实环境部署案例研究

如图24和图25所示，我们搭建了一个真实的家庭环境，其中包含多款智能家电，包括灯、门锁、空气净化器、加湿器和台灯。我们使用Home Assistant平台 [4]通过MQTT消息来控制这些来自不同制造商和品牌的智能家电。如图24所示，Home Assistant平台、MQTT代理以及我们的VCU-LLM均部署在本地边缘设备上。

举一个具体的端到端示例，当用户输入指令“这里太干燥了”时，模糊指令知识检索算法首先将控制因子识别为“湿度”，然后确定相关设备为加湿器。它使用选定的设备及其当前状态构建输入提示，随后输入到本地大语言模型以生成控制计划：“*humidifier.livingroom(开启; 设置\_风扇\_level(3))*”。该控制计划通过MQTT代理转发至Home Assistant平台。相应的自动化控制脚本已预先部署在Home Assistant平台上。当收到控制指令时，它们会向设备的云物联网平台（例如小米物联网平台 [8]）发起HTTP请求，以控制加湿器并

返回更新后的 将状态更新至Home Assistant平台。更新后的加湿器状态随后与 th  
输入提示，作为 VCU-LLM中本地大语言模型的输入，用于后续的模糊指令处理。

在实验中，我们测试了10条不同的指令（包括无关指令）各三次，每次设备都能根据VCU-LLM生成的控制计划准确改变其状态。对于可执行指令，从指令输入到设备控制完成的平均延迟为5.69秒（人工记录，从指令输入完成到观察到智能家居设备状态变化为止）。从生成控制计划到最终执行设备控制的整体延迟大约在1.0秒以内，这表明与家庭助手和物联网平台的交互既稳健又快速。由于大部分开销消耗在设备端大语言模型上，如果家庭中的设备数量减少，整体延迟有望进一步降低。

在实际使用中，设备可能会发生变化（例如，可能添加新设备，旧设备可能离线），或者设备状态可能被手动修改。Home Assistant平台可以及时接收这些更新。我们使用Home Assistant平台上的脚本，通过MQTT代理将这些更新后的信息发送到我们边缘设备上的VCU-LLM。如果仅仅是设备状态发生变化，设备端大语言模型内的预填充键值缓存无需修改。然而，如果设备被添加或移除，则提示中的服务信息以及预填充键值缓存都需要更新（通常耗时少于5秒）。总之，在任何设备变更之后，VCU-LLM都会及时与Home Assistant平台同步，通知用户变更，并根据最新的可用设备生成控制计划。

我们还进行了额外的实验，以进一步调查用户在真实世界部署的VCU-LLM系统中的用户体验。八名测试用户被允许在家庭环境中自由活动30分钟，并鼓励他们积极与VCU-LLM交互以控制智能家电。我们记录了测试用户与VCU-LLM之间的所有交互。在会话结束时，我们采访了测试用户，收集他们对系统的见解和反馈。下面，我们总结了几项关键观察：

- **所有用户都认为，通过模糊指令控制智能家居家电设备能显著减轻用户负担，并且他们对VCU-LLM普遍表示高度满意。**此外，所有用户都认为VCU-LLM的延迟是可接受的。
- **当用户需要同时控制多个智能家电，或者不确定目标家电设备的当前状态时，他们往往更倾向于使用模糊指令。**例如，当用户准备睡觉或离开家时，他们不会明确指定每个单独设备的状态；相反，发出模糊指令更为方便。在八位测试用户提供的全部42条命令中，有28条是模糊指令（占66.6%），这证明了支持模糊指令理解对于家电控制的必要性。
- **部分模糊指令的控制结果未能完全满足用户，但仍被认为是有帮助的。**对于具有强主观性的指令（例如“我们来开个派对吧”），由此产生的设备状态变化可能无法符合每位用户的预期。然而，大多数家电的控制结果是合理的，用户可以通过对单个智能家电进行微小的后续调整，来达到他们期望的控制结果。
- **在未安装人体传感器的情况下，当用户不在主要区域时，最好指定房间（使用明确或模糊指令）以实现更精确的设备控制。**本地大语言模型默认将客厅视作主要区域。如果家庭布局中不包含客厅，本地大语言模型将根据当前上下文推断最可能的主要区域，或在某些情况下将所有房间视为目标。

## 6 相关工作

除了第1节和第2节讨论的大多数相关工作外，我们也对其他相关研究进行了概述。

**大语言模型中的检索增强生成（RAG）。**RAG [21, 27, 41] 是解决大语言模型幻觉问题的一种主流方法。当用户输入一个查询时，它首先被转换为词嵌入，然后用于从预构建的向量索引中检索最相关的文档或文本。检索到的结果随后与输入提示拼接，以生成更可靠、更准确的响应。本文提出的模糊指令知识检索算法与RAG技术有着相似的目标，旨在提升大语言模型输出内容的质量。关键区别在于我们的算法专门针对基于用户模糊指令的智能家居设备控制场景而设计。它同样足够轻量级，能够在资源受限的边缘设备上高效运行（延迟低于100毫秒）。

**基于大语言模型的智能体。**当一个复杂任务需要分解为多个步骤，或者需要与外部系统和工具集成时，基于大语言模型的智能体通常被证明是有效的。随着近期的研究 [23, 34, 43]，智能体已展现出自动化流程、简化工作流程并显著提高效率的能力。一些工作也专注于开发 [26, 37, 50] 用于智能家居环境的智能体，以增强用户与智能设备的交互。然而，这些基于大语言模型的智能体或控制系统通常需要多次调用大语言模型，并依赖大规模大语言模型来支持任务规划、用户意图理解和工具使用等能力，这使得它们难以部署在边缘设备上。相比之下，VCU-LLM首先应用模糊指令知识检索算法，以最低成本生成一个精炼的输入提示。随后，一个在定制的模糊指令数据集上微调后的轻量级大语言模型可以直接生成可执行且合理的控制计划。整个过程在边缘设备本地以低延迟执行，这极大地促进了基于大语言模型智能家居控制系统的实际部署。

## 7 讨论

**通用性。**VCU-LLM 是首个用于智能家居环境中设备端模糊指令理解和设备控制计划生成的方法。它能适应不同的家居布局、设备类型和当前状态，根据用户当前的模糊指令生成合适的控制计划。我们的模糊指令数据集包含 70 个常见房间名称和超过 2000 条不同的模糊指令。将房间名称替换为任意标识符（例如“A1”）不会影响正确控制计划的生成（例如“light.A1(on)”）。在自由交互案例研究中，用户反馈 VCU-LLM 对于无关指令以及所有与控制相关的指令都具有足够的鲁棒性。当用户尝试控制家中不存在的设备，或者当请求的操作无法通过可用设备实现时，VCU-LLM 会返回“None”结果。然而，当家中存在模糊指令数据集未涵盖的设备类型时，生成的控制计划可能会忽略这些设备或产生错误的设备状态。部署 VCU-LLM 后，可以根据用户反馈收集额外的模糊指令示例、设备类型和新的服务信息，以增强模糊指令知识检索算法，并用扩展的数据集对大语言模型进行微调，从而持续提升泛化能力。VCU-LLM 也能为显式指令生成正确的控制计划。当接收到一条指令时，我们首先使用一个轻量级词性分析模型来提取设备名称、房间和目标状态，以判断该指令是否为显式指令。如果被识别为显式指令，则可以直接将其转换为可执行的控制计划，这极大地减少了计算开销。

**真实环境部署。**由VCU-LLM生成的控制计划需要被转换为Home Assistant平台可识别的格式，并需要设置相应的自动化规则。在给定所有可用家电设备及其房间信息的情况下，此转换可通过Python脚本自动完成，这极大地减轻了用户的负担。如图24所示，在实际执行设备控制指令时，仍然需要访问云物联网平台，并未实现完全本地化。这是因为

支持Home Assistant本地控制功能的内置中央集线器网关的固件尚未由制造商发布。即便如此，VCU-LLM仍显著降低了云负载，降低了响应延迟，并为用户隐私提供了保护。未来，一旦实现完全本地化，智能家居模糊指令控制的整体延迟将进一步降低。在真实的家庭环境中，多个用户可能向VCU-LLM发出冲突的指令。然而，与触发-动作程序生成工作不同 [26, 52, 64]，——在那些工作中，来自不同用户的预定义冲突TAP可能在触发器激活后导致家电设备无法被正确控制（例如，灯被要求同时开启和关闭）——实时模糊指令控制将每条指令视为一个可执行的用户请求。因此，我们只需按照接收顺序依次执行这些指令。

**局限性。**尽管 VCU-LLM 能够为模糊指令、显式指令以及无关指令生成适当的输出，但在处理与家庭助手交互无关的较长文本（例如任意一段文本）时，其输出可能变得不稳定。目前，VCU-LLM 主要支持英语，因为轻量级大语言模型的有限能力使得在单一模型中处理多种语言具有挑战性。未来，我们计划通过翻译训练样本并更新模糊指令知识检索算法中的组件（例如句子转换器、词性标注模型、关键词等）的不同版本来扩展对其他语言的支持。

**多轮交互。**无论是用于大语言模型微调的模糊指令知识检索还是模糊指令数据集，都是为单轮交互设计的，这当用户对控制结果不满意并希望调整时引入了限制。然而，有几点需要澄清。(1) 在我们的案例研究中，此类情况很少发生，并且用户并未表现出对多轮对话的明确需求，因为 VCU-LLM 的设计目标是在单次交互中完全解读用户通过模糊指令表达的控制意图——这本身就是在低延迟的边缘设备上执行已具挑战性的任务。(2) 即使对于在基于人类的评估中获得较低分数的模糊指令，用户通常也只需要调整特定设备的单一状态。在这种情况下，多轮交互并不会减轻用户负担。例如，当用户对灯光亮度不满意时，说“房间太亮了”已经有效，相比于多轮交互如“不，灯光亮度设置得太高了”。(3) 多轮对话并不能有效减少纠正目标设备所需的努力。例如，回应“不，不是那盏灯”并不能让系统识别出正确的控制目标；仍然需要额外信息（例如“是卧室里的灯”），这本质上等同于发出一个显式指令，如“打开卧室的灯”。(4) 在资源受限的边缘设备上部署多轮交互本身就具有挑战性。维护对话历史会产生大量的内存和计算开销，这对于上下文窗口长度有限的轻量化大语言模型来说是难以承受的。此外，在严格的资源预算下，确定可以保留和访问多少过去的交互本身也非易事。(5) 一种可能的方法是首先使用模糊指令数据集中的单轮数据进行指令微调，然后使用少量高质量的多轮对话数据来增强对话连贯性。我们将对此方向的系统性探索留作未来的工作。

## 8 结论

在本工作中，我们提出了VCU-LLM，这是首个在边缘设备上部署大语言模型以实现模糊指令理解和智能设备控制计划生成的系统。VCU-LLM将生成的控制计划质量平均提升了43.3%，同时将时间开销平均降低了8.44 ×。此外，VCU-LLM具有很强的通用性，能以极低成本部署到不同家庭，并与其他智能家居平台保持良好的兼容性。

## 致谢

本工作部分得到了中国国家自然科学基金（项目号62572069和62232004）、111计划（项目号B18008）以及北京邮电大学创新创业支持计划（项目号2025-YC T014）的资助。赵东是通讯作者。

## 参考文献

- [1] 2025. Alexa智能家居 – 亚马逊. <https://www.amazon.com/alexa-smart-home/>. [2] 2025. all-MiniLM-L6-v2. <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>. [3] 2025. GGUF. <https://huggingface.co/docs/hub/en/gguf>. [4] 2025. Home Assistant. <https://www.home-assistant.io/>. [5] 2025. Home-Assistant-Requests. <https://huggingface.co/datasets/acon96/Home-Assistant-Requests>. [6] 2025. HomeLLM. <https://github.com/acon96/home-llm>. [7] 2025. Llama.cpp. <https://github.com/ggml-org/llama.cpp>. [8] 2025. 小米物联网平台. [https://github.com/XiaoMi/ha\\_xiaomi\\_home](https://github.com/XiaoMi/ha_xiaomi_home). [9] 2025. 全球智能家居用户数量从2019年到2028年. <https://www.statista.com/forecasts/887613/number-of-smart-homes-in-the-smart-home-market-in-the-world>. [10] 2025. OpenAI. <https://openai.com/>. [11] 2025. OpenAI API. <https://openai.com/api/>. [12] 2025. 树莓派5. <https://www.raspberrypi.com/products/raspberry-pi-5/>. [13] 2025. 智能家居 – 全球. <https://www.statista.com/outlook/cmo/smart-home/worldwide>. [14] 2025. 智能家居 | 小米全球. <https://www.mi.com/global/smart-home/>. [15] 2025. 智能家居规范. <https://home.miot-spec.com/>. [16] 2025. 智能音箱消费者采用报告2018. <https://voicebot.ai/download-smart-speaker-consumer-adoption-report-2018/>. [17] 2025. 欢迎来到一个更有帮助的家 – Google Home. <https://home.google.com/welcome/>. [18] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, 等人. 2023. GPT-4技术报告. arXiv预印本 arXiv:2303.08774 (2023). [19] Keivan Alizadeh, Seyed Iman Mirzadeh, Dmitry Belenko, S Khatamifard, Minsik Cho, Carlo C Del Mundo, Mohammad Rastegari, Mehrdad Farajtabar. 2024. 闪存中的大语言模型：有限内存下的高效大语言模型推理. 在《第62届计算语言学协会年会论文集》(ACL). 12562–12584. [20] Jimmy Lei Ba, Jamie Ryan Kiros, Geoffrey E Hinton. 2016. 层归一化. arXiv预印本 arXiv:1607.06450 (2016). [21] Qinyuan Cheng, Xiaonan Li, Shimin Li, Qin Zhu, Zhangyue Yin, Yunfan Shao, Linyang Li, Tianxiang Sun, Hang Yan, Xipeng Qiu. 2024. 检索增强生成的统一主动检索. arXiv预印本 arXiv:2406.12534 (2024). [22] Benjamin R Cowan, Nadia Pantidi, David Coyle, Kellie Morrissey, Peter Clarke, Sara Al-Shehri, David Earley, Natasha Bandeira. 2017. “我能帮您做什么？”不常用用户对智能个人助理的体验. 在《第19届移动设备与服务人机交互国际会议论文集》(MobileHCI). 1–12. [23] Shihan Deng, Weikai Xu, Hongda Sun, 刘伟, Tao Tan, Jianfeng Liu, Ang Li, 栾健, 王斌, Rui Yan, 等人. 2024. Mobile-bench: 基于大语言模型的移动智能体评估基准. arXiv预印本 arXiv:2407.00993 (2024). [24] Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova. 2018. BERT: 用于语言理解的深度双向Transformer预训练. 在 arXiv预印本 arXiv:1810.04805. 1–16. [25] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, 等人. 2024. Llama 3模型群. arXiv电子版 (2024), arXiv-2407. [26] Yi Gao, Kaijie Xiao, Fu Li, Weifeng Xu, Jiaming Huang, Wei Dong. 2024. ChatIoT: 基于触发-动作的物联网程序零代码生成. 《ACM交互式、移动、可穿戴和普适技术汇刊》8, 3 (2024), 1–29. [27] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, Haofen Wang. 2023. 大语言模型的检索增强生成：综述. arXiv预印本 arXiv:2312.10997 2, 1 (2023). [28] Radhika Garg, Subhasree Sengupta. 2020. 他就像我一样：父母与儿童长期使用智能音箱的研究. 《ACM交互式、移动、可穿戴和普适技术汇刊》4, 1 (2020), 1–24. [29] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, 等人. 2025. Deepseek-r1: 通过强化学习激励大语言模型的推理能力. arXiv预印本 arXiv:2501.12948 (2025年). [30] Liwei Guo, Wonkyo Choe, Felix Xiaozhu Lin. 2023. STI: 通过弹性流水线在边缘为自然语言处理推理加速. 在《第28届ACM编程语言与操作系统架构支持国际会议论文集, 第2卷》. 791–803. [31] Matthew Honnibal, Ines Montani. 2017. spaCy 2: 使用Bloom嵌入、卷积神经网络和增量解析的自然语言理解. Zenodo软件发布. doi:10.5281/zenodo.1212303

- [32] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, 等人. 2022. LoRA: 大语言模型的低秩适应. ICLR 1, 2 (2022), 3.[33] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, 等人. 2025. 大语言模型中的幻觉综述：原理、分类、挑战与开放性问题. 《ACM信息系统汇刊》43, 2 (2025年), 1–55. [34] Xu Huang, Weiwen Liu, Xiaolong Chen, Xingmei Wang, Hao Wang, Defu Lian, Yasheng Wang, Ruiming Tang, Enhong Chen. 2024. 理解LLM智能体的规划：综述. arXiv预印本 arXiv:2402.02716 (2024).[35] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, Dario Amodei. 2020. 神经语言模型的缩放定律. arXiv预印本 arXiv:2001.08361 (2020). [36] Sunyoung Kim, Abhishek Choudhury. 2021. 探索老年人对基于智能音箱的语音助手的感知与使用：一项纵向研究. 《计算机与人类行为》(COHB) 124 (2021), 106914. [37] Evan King, Haoxiang Yu, Sangsu Lee, Christine Julien. 2023. “准备派对”：借助大语言模型探索更智能的智能空间. arXiv预印本 arXiv:2303.14143 (2023). [38] Evan King, Haoxiang Yu, Sangsu Lee, Christine Julien. 2024. Sasha：智能家居中大语言模型的创造性目标导向推理. 《ACM交互式、移动、可穿戴和普适技术汇刊》8, 1 (2024), 1–38. [39] Brian Lester, Rami Al-Rfou, Noah Constant. 2021. 规模对参数高效提示调优的力量. arXiv预印本 arXiv:2104.08691 (2021).[40] Yaniv Leviathan, Matan Kalman, Yossi Matias. 2023. 通过推测解码实现Transformer的快速推理. 在《国际机器学习大会》(ICML). 19274–19286. [41] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, 等人. 2020. 知识密集型自然语言处理任务的检索增强生成. 《神经信息处理系统进展》33 (2020), 9459–9474. [42] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, Deming Chen. 2024. Snapkv：大语言模型在生成前就知道你在寻找什么. 《神经信息处理系统进展》37 (2024), 22947–22970.[43] Yuanchun Li, Hao Wen, Weijun Wang, Xiangyu Li, Yizhen Yuan, Guohong Liu, Jiacheng Liu, Wenxing Xu, Xiang Wang, Yi Sun, 等人. 2024. 个人LLM智能体：关于能力、效率与安全的见解与综述. arXiv预印本 arXiv:2401.05459 (2024). [44] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, Xia Hu. 2024. KIVI：一种用于KV缓存的免调优非对称2比特量化. arXiv预印本 arXiv:2402.02750 (2024). [45] Sudipta Paul, Lingyu Zhang, Yilin Shen, Hongxia Jin. 2024. 赋能小语言模型的设备控制规划能力. 在《ICASSP 2024-2024 IEEE声学、语音与信号处理国际会议》(ICASSP). 12066–12070. [46] Alisha Pradhan, Amanda Lazar, Leah Findlater. 2020. 低技术使用率的老年人对智能语音助手的使用. 《ACM人机交互汇刊》(TOCHI) 27, 4 (2020), 1–27. [47] Nils Reimers, Iryna Gurevych. 2019. Sentence-BERT：使用孪生BERT网络的句子嵌入. arXiv预印本 arXiv:1908.10084 (2019). [48] Shiya Ren, Huaming Wang, Dongming Yu, Yuan Li, Zhixing Li, S Hu, L Zou. 2018. 基于规则的联合意图检测与槽填充. CCKS任务 2242 (2018), 34–40. [49] Jordan Rey-Jouanchicot, André Bottaro, Eric Campo, Jean-Léon Bouraoui, Nadine Vigouroux, Frédéric Vella. 2024. 利用大语言模型增强智能家居的个性化用户体验. arXiv预印本 arXiv:2407.12024 (2024). [50] Dmitriy Rivkin, Francois Hogan, Amal Feriani, Abhishek Konar, Adam Sigal, Xue Liu, Gregory Dudek. 2024. 通过自主LLM智能体实现的AIoT智能家居. 《IEEE物联网期刊》(2024).[51] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, Ce Zhang. 2023. FlexGen：使用单GPU对大语言模型进行高吞吐量生成推理. 在《国际机器学习大会》(ICML). 31094–31116. [52] Yingtian Shi, Xiaoyi Liu, Chun Yu, Tianao Yang, Cheng Gao, Chen Liang, Yuanchun Shi. 2024. 弥合智能家居中自然用户表达与复杂自动化编程之间的鸿沟. arXiv预印本 arXiv:2408.12687 (2024). [53] Qwen团队. 2024. Qwen2.5：基础模型的聚会. <https://qwenlm.github.io/blog/qwen2.5/> [54] SM Tonmoy, SM Zaman, Vinija Jain, Anku Rani, Vipula Rawte, Aman Chadha, Amitava Das. 2024. 大语言模型幻觉缓解技术全面综述. arXiv预印本 arXiv:2401.01313 6 (2024). [55] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, 等人. 2023. Llama：开放且高效的基础语言模型. 在 arXiv预印本 arXiv:2302.13971. 2556–2565.[56] Pooja Upadhyay, Sharon Heung, Shiri Azenkot, Robin N Brewer. 2023. 研究老年人对语音助手的探索与长期使用. 在《2023年CHI计算系统中的人为因素会议论文集》(CHI). 1–11.

[57] Krishna Sruthi Velaga 和 郭一凡。2025年。在边缘优化大语言模型辅助的智能家居助手系统：一项实证研究。发表于 *AAAI 2025* 无线通信与网络人工智能研讨会 (*AI4WCN*)。[58] Henry Weld、黄晓琪、龙思曲、潘乔赛 和 韩素妍·凯伦。2022年。自然语言理解中联合意图检测与槽填充模型综述。《计算机概览》55, 8 (2022), 1–38。[59] Charlie Wilson、Tom Hargreaves 和 Richard Hauxwell-Baldwin。2015年。智能家居及其用户：系统分析与关键挑战。《个人与普适计算》19 (2015), 463–476。[60] 徐大亮、殷望松、张浩、金鑫、张颖、魏世云、徐梦伟 和 刘譞哲。2024年。EdgeLLM：通过推测解码实现快速的设备端LLM推理。《*IEEE*移动计算汇刊》(*TMC*)(2024)。[61] 杨安、李安峰、杨宝松、张北辰、回彬源、郑博、于博文、高畅、黄晨耕、吕晨旭 等。2025年。Qwen3技术报告。*arXiv*预印本 *arXiv:2505.09388*(2025)。[62] 杨敬波、侯柏如、魏伟、鲍宇佳 和 常诗雨。2025年。Kvlink：通过高效的KV缓存复用加速大语言模型。*arXiv*预印本 *arXiv:2502.16002*(2025)。[63] 姚鼎宇、沈博文、林铮、刘伟、栾健、王斌 和 王卫平。2025年。TailorKV：通过定制化KV缓存优化实现长上下文推理的混合框架。*arXiv*预印本 *arXiv:2505.19586*(2025)。[64] 张乐凡、何伟佳、Olivia Morkved、赵薇、Michael L Littman、卢山 和 Blase Ur。2020年。Trace2tap：从行为轨迹合成触发-动作程序。*ACM交互式、移动、可穿戴和普适技术汇刊*4, 3 (2020), 1–26。[65] 张培元、曾广涛、王天铎 和 陆伟。2024年。Tinyllama：一个开源的小语言模型。*arXiv*预印本 *arXiv:2401.02385*(2024)。[66] 郑慧敏、邢晓峰 和 徐祥敏。2024年。SleepCoT：通过思维链蒸馏实现的轻量级个性化睡眠健康模型。*arXiv*预印本 *arXiv:2410.16924*(2024)。